Data Analysis

Lecture notes with R, Lectures 2–11

Contents

Lecture 2. Data Visualization 4

2.1 Why visualise data 4
2.1.1 Example: campaign spending in 2012 . . . 5

2.2 A taxonomy for data graphics 5

2.3 A directory of visualisations 6

2.4 Visualising distributions 6
2.4.1 Histograms and bin width . . . 7
2.4.2 Density estimates . . . 7
2.4.3 Comparing distributions . . . 8

2.5 Visualising associations 8

2.6 Visualising trends 9

2.7 Telling a story 10

Lecture 3. Data Wrangling 12

3.1 A grammar of data wrangling 12
3.1.1 The single-table verbs . . . 12

3.2 Relational data 14

3.3 Example: Uber pickups in New York 15

Lecture 4. Statistical Foundations 17

4.1 From description to statistical methods 17

4.2 Samples and populations: a travel policy 17

4.3 Sampling distributions 18

4.4 The bootstrap 20

4.5 Outliers 21

4.6 Statistical models 22

Lecture 5. Linear Regression **24**

5.1 Simple linear regression **24**
5.1.1 Example: the rail trail . . . 24
5.1.2 Prediction and intervals . . . 25
5.1.3 Strength of fit and least squares . . . 26
5.1.4 A categorical predictor . . . 26

5.2 Loss functions **27**

5.3 Multiple regression **28**
5.3.1 Inference for coefficients . . . 28
5.3.2 Choosing a model . . . 29

5.4 Other considerations **30**

Lecture 6. Generalized Linear Models and Model Evaluation **31**

6.1 Generalized linear models **31**

6.2 Logistic regression **31**
6.2.1 Coefficients as odds ratios . . . 32

6.3 GLMs for counts **32**
6.3.1 Poisson regression . . . 32
6.3.2 Overdispersion and the negative binomial . . . 33
6.3.3 Zero-inflated Poisson . . . 34

6.4 Evaluating models **34**
6.4.1 Training error and the holdout method . . . 35
6.4.2 Cross-validation . . . 35
6.4.3 Why a separate test set . . . 36

Lecture 7. Regularization **37**

7.1 Beyond least squares **37**

7.2 Subset selection **37**
7.2.1 Choosing the optimal model . . . 38

7.3 Regularization **38**
7.3.1 Penalized form . . . 39

7.4 Example: delays of flights to California **40**

Lecture 8. Classification I **43**

8.1 Supervised learning **43**

8.2 Naive Bayes **44**

8.3 Discriminant analysis **44**

8.4 Logistic regression for classification **46**
8.4.1 Why not squared loss . . . 46
8.4.2 Cross-entropy loss . . . 47

Lecture 9. Classification II **49**

9.1 Logistic regression as a classifier **49**
9.1.1 Thresholding . . . 49
9.1.2 Accuracy . . . 49

9.2 Evaluating classifiers **50**
9.2.1 Drawbacks of accuracy . . . 50
9.2.2 The confusion matrix . . . 50
9.2.3 ROC and precision–recall curves . . . 51
9.2.4 Training versus testing . . . 51

9.3 Linear separability and regularization **52**

Lecture 10. Splines and Generalized Additive Models **54**

10.1 Beyond linearity **54**

10.2 Polynomial regression and step functions **54**

10.3 Regression splines **55**

10.4 Smoothing splines **57**

10.5 Local regression **58**

10.6 Generalized additive models **58**

Lecture 11. Causal Inference **61**

11.1 Why causation **61**

11.2 Potential outcomes **62**

11.3 Randomised experiments and observational studies **62**
11.3.1 Unconfoundedness . . . 63

11.4 Propensity score methods **63**
11.4.1 Stratification . . . 64
11.4.2 Matching . . . 64
11.4.3 Inverse probability weighting . . . 64

11.5 Example: job training and earnings **65**

Lecture 2. Data Visualization

These notes follow the course slides (based on *Modern Data Science with R*, Baumer, Kaplan and Horton, and Wilke's *Fundamentals of Data Visualization*). All R code was run when the notes were compiled; lines starting with `##` are output. Data sets come from R packages so that every example can be reproduced; where the slides used other data, the notes say so. Material marked [Supplement] is not on the slides.

2.1 Why visualise data

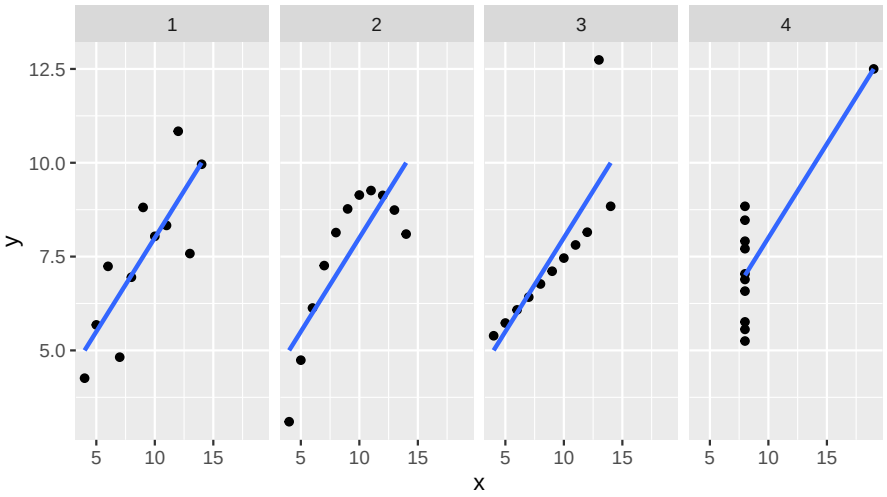
Motivation. Summary statistics compress a data set into a few numbers, and very different data sets can share the same numbers. A plot shows the structure that the numbers hide, which is why visualisation is part of analysis rather than decoration.

Visualisation lets us explore data and find patterns that would be missed if we relied only on numerical summaries. The classic illustration is **Anscombe's quartet**: four data sets with (almost) identical means, variances, correlations and regression lines.

```
anscombe_long <- anscombe %>%
  pivot_longer(everything(), names_to = c(".value", "set"), names_pattern = "(.)(.)")
anscombe_long %>%
  group_by(set) %>%
  summarise(mean_x = mean(x), mean_y = mean(y), sd_y = sd(y), cor = cor(x, y))
```

```
## # A tibble: 4 x 5
##   set   mean_x mean_y  sd_y   cor
##   <chr>  <dbl>  <dbl> <dbl> <dbl>
## 1 1          9   7.50  2.03 0.816
## 2 2          9   7.50  2.03 0.816
## 3 3          9   7.5   2.03 0.816
## 4 4          9   7.50  2.03 0.817
```

```
ggplot(anscombe_long, aes(x, y)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE, formula = y ~ x) +
  facet_wrap(~ set, nrow = 1)
```



Only the first set looks like the textbook picture of a linear relationship: the second is a curve, the third has one outlier, and the fourth is determined by a single point. Visualisation "forces us to notice what we never expected to see" (Tukey, 1977).

There is no fixed procedure. Any data set can be visualised in many ways, some informative and some not; we should look at the data from different perspectives and at coarser and finer levels, starting simply and refining by trial and error. Tufte's guiding principles: above all show the data; avoid distorting what the data have to say; present many numbers in a small space; encourage the eye to compare; reveal the data at several levels of detail.

The main tools are histograms, density plots and bar plots (distributions of continuous and discrete variables), boxplots (distributions, tied to robust summaries), and scatterplots and their variants (bivariate data); heat maps and maps serve more specialised purposes.

2.1.1 Example: campaign spending in 2012

The slides analyse US Federal Election Commission data on spending in the 2012 elections. Three lessons from that example:

- **A third variable can reverse the story.** Total spending on each candidate made Obama look like the best-funded candidate, but colouring by type of spending showed that $251 million of the $274 million spent on his campaign was spent *against* him (92%, against 70% for Romney).
- **Faceting adds context.** Splitting spending by office (House, Senate, President) showed that only in the presidential race was more money spent attacking Democrats than Republicans.
- **Proportions, not totals.** Relating money to votes in House races, the vote share should be compared with the *share* of spending, since districts differ in population and turnout. Skewed variables (donation amounts) are better seen on a log scale, and distributions are easier to compare overlaid or stacked than side by side.

2.2 A taxonomy for data graphics

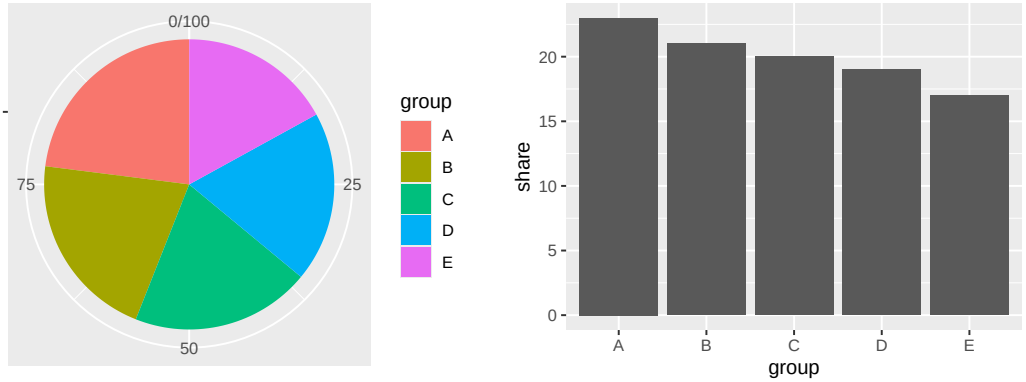
Motivation. To design a graphic deliberately, and to criticise one, it helps to break it into parts. Yau's taxonomy of four elements, which matches the grammar of graphics implemented in ggplot2, does this.

Every data graphic can be described by four elements (Yau, 2013):

1. **Visual cues**: graphical elements that draw the eye. In decreasing order of how accurately people perceive differences: position, length, angle, direction, shape (categorical), area, volume, shade and colour.
2. **Coordinate system**: Cartesian, polar, or geographic.
3. **Scale**: how distance in the graphic translates into differences in quantity. Numeric scales may be linear, logarithmic or percentage; categorical scales may be unordered or ordinal; time is numeric but has calendar units and can be periodic.
4. **Context**: titles, axis labels, reference points or lines that help the viewer make meaningful comparisons.

Consequences of the perception ranking: people judge positions and lengths well but angles and areas poorly, so **bar charts beat pie charts** and word clouds or bubble sizes convey magnitudes only roughly. Differences in colour are also hard to judge (heat maps), and red–green contrasts fail for colour-blind viewers.

```
shares <- tibble(group = LETTERS[1:5], share = c(23, 21, 20, 19, 17))
ggplot(shares, aes(x = "", y = share, fill = group)) +
  geom_col(width = 1) + coord_polar(theta = "y") + labs(x = NULL, y = NULL)
ggplot(shares, aes(group, share)) + geom_col()
```



The same five shares: in the pie the slices look equal, in the bar chart the ordering is obvious.

Dissecting a graphic. For any data graphic ask: which variables are shown and of what type; which visual cue encodes each; which coordinate system and scales are used; how context is provided. For example, a bar chart of average math SAT scores by state shows a continuous variable (score) and a categorical one (state) using length, in Cartesian coordinates with linear scales, with context from axis labels and a title.

2.3 A directory of visualisations

Motivation. Most plotting questions fall into a few types: amounts, distributions, proportions, relationships, trends and geography. Knowing the standard plot for each type is the quickest route to a good first graphic.

what to show	plots
amounts	bars (vertical or horizontal), dots where the bar would end
one distribution	histogram, density (need a binwidth or bandwidth); empirical CDF, QQ plot (faithful but harder to read)
several distributions	boxplots, violins, overlapping densities, ridgeline plots (many groups)
proportions	side-by-side bars, stacked bars (many conditions), pies (only for simple fractions of a whole); stacked densities when proportions change along a continuous variable
multiple groupings	mosaic plots, treemaps, parallel sets
x–y relationships	scatterplots, bubble charts (third variable as size), 2-d bins for many points, correlograms for many variables, line graphs when x is time
geospatial	choropleth maps, cartograms

2.4 Visualising distributions

Motivation. A histogram or density estimate depends on a tuning choice, the bin width or bandwidth, and different choices can tell different stories. Knowing the standard rules gives a principled starting point.

2.4.1 Histograms and bin width

A histogram depends on the bin width h: too narrow and noise dominates, too wide and structure disappears (the slides show Titanic passenger ages with widths of 1, 3, 5 and 15 years). Two common rules:

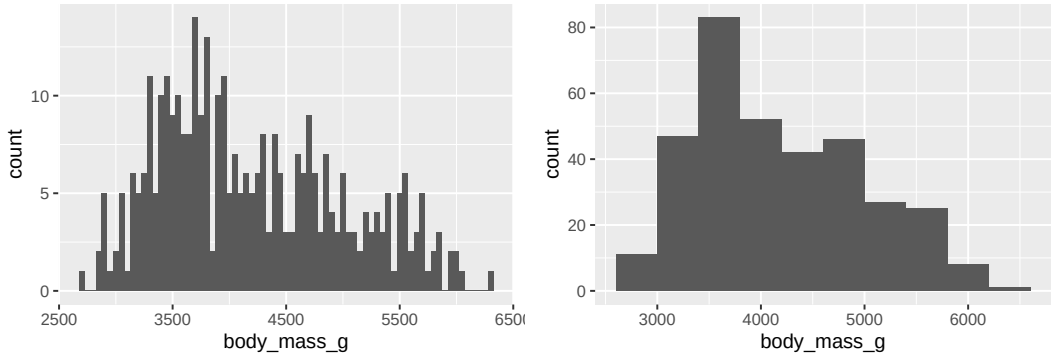
$$\text{Scott: } h = 3.49\,\hat{\sigma}\,n^{-1/3}, \qquad \text{Freedman–Diaconis: } h = 2\,\mathrm{IQR}\,n^{-1/3},$$

the second being less sensitive to outliers. For penguin body mass:

```
x <- na.omit(penguins$body_mass_g)
c(n = length(x), scott = 3.49 * sd(x) * length(x)^(-1/3),
  fd = 2 * IQR(x) * length(x)^(-1/3))
```

```
##        n    scott       fd
## 342.0000 400.2209 343.1910
```

```
ggplot(penguins, aes(body_mass_g)) + geom_histogram(binwidth = 50, na.rm = TRUE)
ggplot(penguins, aes(body_mass_g)) + geom_histogram(binwidth = 400, na.rm = TRUE)
```



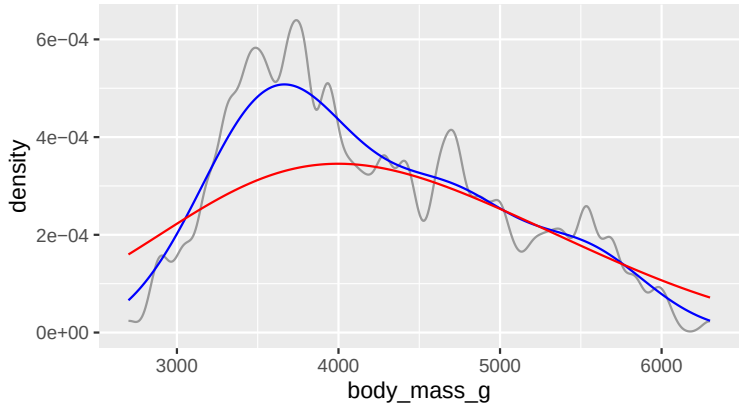
2.4.2 Density estimates

A kernel density estimate places a small bump (the **kernel** K) on each observation and adds them up:

$$\hat{f}(x) = \frac{1}{nh}\sum_{i=1}^{n} K\Big(\frac{x - X_i}{h}\Big).$$

It depends on the kernel and, much more, on the bandwidth h. A rule of thumb (Härdle, 1991) is $h = 1.06\,S\,n^{-1/5}$, where S is the standard deviation; robust versions replace S by the scaled IQR $(X_{(0.75n)} - X_{(0.25n)})/1.34$ or the scaled MAD $1.4826 \cdot \mathrm{median}|X_i - \mathrm{median}(X)|$.

```
ggplot(penguins, aes(body_mass_g)) +
  geom_density(bw = 50, colour = "grey60", na.rm = TRUE) +
  geom_density(bw = 1.06 * sd(x) * length(x)^(-1/5), colour = "blue", na.rm = TRUE) +
  geom_density(bw = 800, colour = "red", na.rm = TRUE)
```



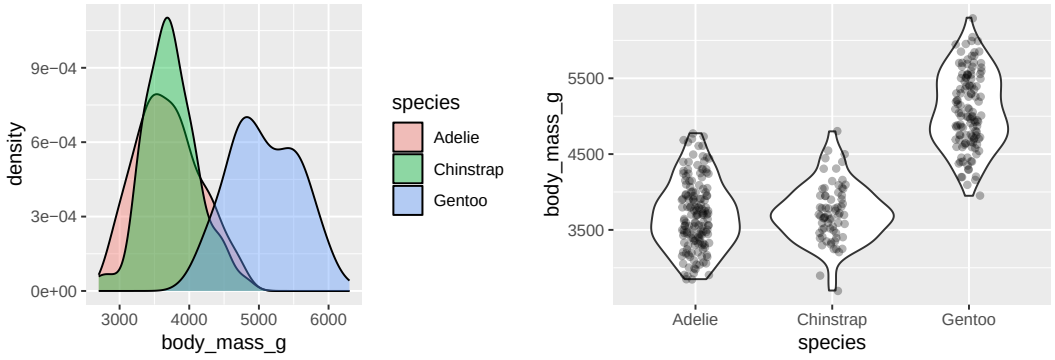
Where the 1.06 comes from [Supplement]

For a normal density with standard deviation σ and a Gaussian kernel, the bandwidth minimising the asymptotic mean integrated squared error is $h^* = (4/3)^{1/5}\sigma n^{-1/5} \approx 1.06\,\sigma n^{-1/5}$. The rule is therefore optimal for normal data and oversmooths multimodal data, as the blue curve above (which blurs the species) shows.

2.4.3 Comparing distributions

For two distributions, overlay densities or histograms (with transparency) rather than placing them side by side. For several, use jittered points, boxplots, violins (mirrored densities) or ridgeline plots (densities staggered vertically, good for many groups; package `ggridges`).

```
ggplot(penguins, aes(body_mass_g, fill = species)) +
  geom_density(alpha = 0.4, na.rm = TRUE)
ggplot(penguins, aes(species, body_mass_g)) +
  geom_violin(na.rm = TRUE) + geom_jitter(width = 0.1, alpha = 0.3, na.rm = TRUE)
```

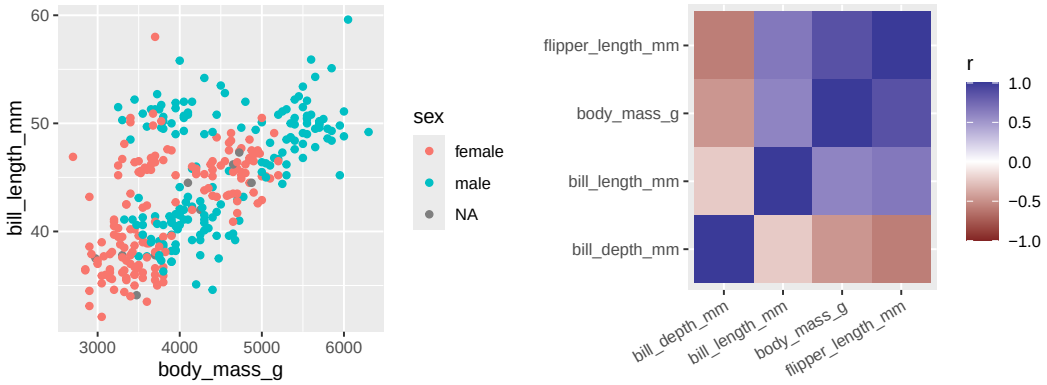


2.5 Visualising associations

The slides use 123 blue jays with head length, skull size and body mass. With `penguins` the same steps are: a scatterplot for two variables; colour for a grouping variable; size for a third quantitative variable (a **bubble chart**, which makes associations hard to judge); a scatterplot matrix of all pairs; and a **correlogram**, a picture of the correlation matrix.

```
ggplot(penguins, aes(body_mass_g, bill_length_mm, colour = sex)) +
  geom_point(na.rm = TRUE)
penguins %>%
```

```
  select(bill_length_mm, bill_depth_mm, flipper_length_mm, body_mass_g) %>%
  cor(use = "complete.obs") %>%
  as_tibble(rownames = "x") %>%
  pivot_longer(-x, names_to = "y", values_to = "r") %>%
  ggplot(aes(x, y, fill = r)) + geom_tile() +
  scale_fill_gradient2(limits = c(-1, 1)) +
  theme(axis.text.x = element_text(angle = 30, hjust = 1)) + labs(x = NULL, y = NULL)
```



`GGally::ggpairs()` draws the full scatterplot matrix in one call.

Simpson's paradox in the penguins [Supplement]

Pooled over species, bill depth and bill length are negatively correlated; within each species the correlation is positive. A grouping variable can reverse an association, which is exactly why colour or facets for a third variable matter.

```
penguins %>% drop_na() %>%
  summarise(pooled = cor(bill_length_mm, bill_depth_mm))
```

```
## # A tibble: 1 x 1
##   pooled
##    <dbl>
## 1 -0.229
```

```
penguins %>% drop_na() %>% group_by(species) %>%
  summarise(within = cor(bill_length_mm, bill_depth_mm))
```

```
## # A tibble: 3 x 2
##   species   within
##   <fct>      <dbl>
## 1 Adelie     0.386
## 2 Chinstrap  0.654
## 3 Gentoo     0.654
```

2.6 Visualising trends

Motivation. Time series and other ordered data mix long-term trends with short-term noise. Smoothing separates them, and a transformation can turn a curved trend into a straight one that is easy to read.

Defined functional forms. The slides show monthly preprint submissions to bioRxiv growing exponentially, $y = Ae^{mx}$. Rather than fitting an exponential curve, take logs: $\log y = \log A + mx$ is linear, so fit a line to the log-transformed data.

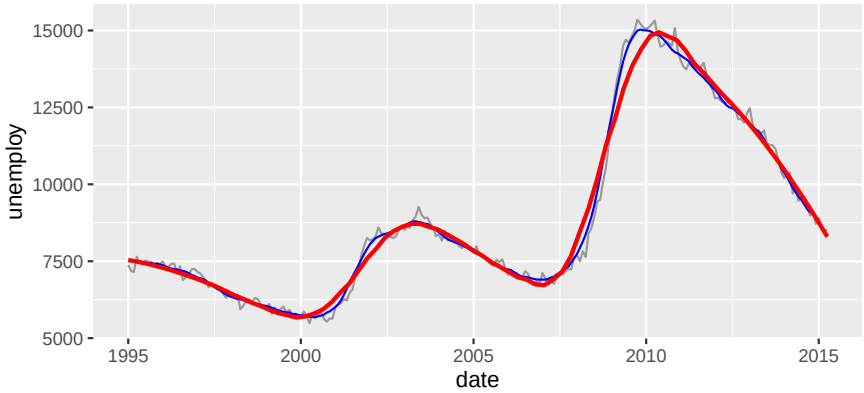
Smoothing. To show a long-term trend while de-emphasising fluctuations (the slides use the Dow Jones index in 2009), use

- a **moving average**: the mean over a window of neighbouring times;
- **LOESS** (locally estimated scatterplot smoothing): at each x_0, take the k nearest observations $N_k(x_0)$ and fit a line $\ell(x) = \beta_0 + \beta_1(x - x_0)$ by weighted least squares, minimising

$$\sum_{i : X_i \in N_k(x_0)} \{Y_i - \ell(X_i)\}^2 \, W\Big(\frac{|x_0 - X_i|}{\Delta(x_0)}\Big),$$

  where $\Delta(x_0) = \max_{i : X_i \in N_k(x_0)} |X_i - x_0|$ and $W(u) = (1 - u^3)^3$ (tri-cube weight). The **span** k/n controls smoothness; a quadratic can replace the line;
- **splines**: piecewise polynomials joined smoothly at *knots*; faster than LOESS for large data, but different spline types and numbers of knots can give quite different curves (Lecture 10).

```
econ <- economics %>%
  filter(date >= as.Date("1995-01-01")) %>%
  mutate(ma12 = stats::filter(unemploy, rep(1/12, 12), sides = 2))
ggplot(econ, aes(date, unemploy)) +
  geom_line(colour = "grey60") +
  geom_line(aes(y = ma12), colour = "blue", na.rm = TRUE) +
  geom_smooth(method = "loess", span = 0.3, se = FALSE, colour = "red", formula = y ~ x)
```



US unemployment (thousands, `ggplot2::economics`): raw series (grey), 12-month moving average (blue) and LOESS with span 0.3 (red).

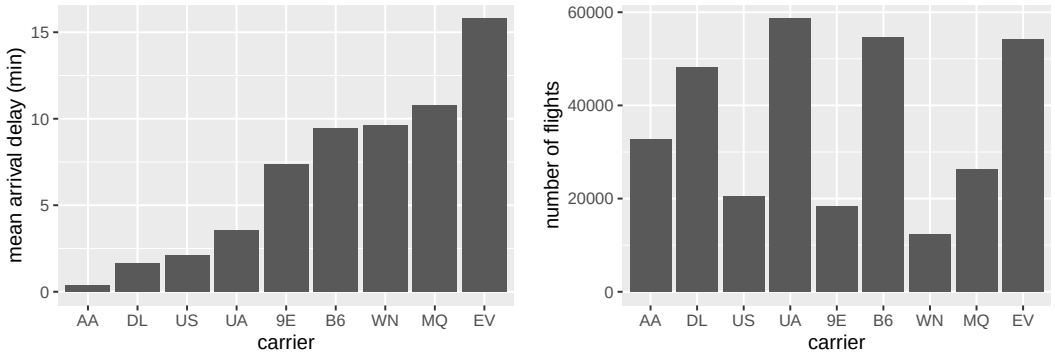
2.7 Telling a story

Motivation. A graphic made for a decision must make its point to people who did not do the analysis. The Challenger disaster is the classic case of the right data shown the wrong way.

On 27 January 1986 engineers at Morton Thiokol recommended delaying the launch of the space shuttle Challenger, fearing that the cold forecast would compromise the rubber O-rings sealing the solid rocket motors. Their 13 charts did not persuade NASA, the launch went ahead, and the O-rings failed 73 seconds after launch, killing seven astronauts. The charts showed O-ring damage for each previous flight but did not plot damage against temperature, and the hearings afterwards showed only flights with damage. Tufte's reconstruction plots damage against launch temperature for *all* flights: every launch below about 65°F had damage, and the forecast temperature (about 30°F) lay far below every previous launch, an extrapolation that by itself argued against launching.

Making a point simply. A plot of mean arrival delay for each carrier departing New York over time is rich but cluttered. If the message is "American and Delta have the shortest delays", a bar chart conveys it directly, and a second bar chart of flight counts shows that both are major NYC carriers:

```
by_carrier <- flights %>%
  group_by(carrier) %>%
  summarise(delay = mean(arr_delay, na.rm = TRUE), n = n()) %>%
  filter(n > 10000)
ggplot(by_carrier, aes(reorder(carrier, delay), delay)) + geom_col() +
  labs(x = "carrier", y = "mean arrival delay (min)")
ggplot(by_carrier, aes(reorder(carrier, delay), n)) + geom_col() +
  labs(x = "carrier", y = "number of flights")
```



Lecture 3. Data Wrangling

3.1 A grammar of data wrangling

Motivation. ggplot2 gave a grammar for graphics; dplyr gives one for manipulating data. A handful of verbs, each doing one thing and each taking and returning a data frame, can be chained to answer most questions about a table.

`dplyr` was designed to provide the common data-manipulation tools, to be fast on in-memory data, and to separate the operations from the data source (the same code works on a database). It works on data frames and on **tibbles**, trimmed-down data frames (class `tbl_df`) that print only the first rows and the columns that fit, with their types. The introductory vignette is `vignette("dplyr", package = "dplyr")`.

We use `ggplot2::presidential`:

```
presidential
```

```
## # A tibble: 12 x 4
##   name       start      end        party
##   <chr>      <date>     <date>     <chr>
## 1 Eisenhower 1953-01-20 1961-01-20 Republican
## 2 Kennedy    1961-01-20 1963-11-22 Democratic
## 3 Johnson    1963-11-22 1969-01-20 Democratic
## 4 Nixon      1969-01-20 1974-08-09 Republican
## 5 Ford       1974-08-09 1977-01-20 Republican
## # i 7 more rows
```

```
class(presidential)
```

```
## [1] "tbl_df"     "tbl"        "data.frame"
```

3.1.1 The single-table verbs

- `filter()` selects rows; `select()` selects columns;
- `mutate()` adds columns; `arrange()` reorders rows;
- `summarize()` collapses a data frame to a single row, usually together with `group_by()`.

filter(). The first argument is the data; the second is a logical vector, and only rows where it is `TRUE` are kept. Comparisons use `>`, `>=`, `<`, `<=`, `!=`, `==`; set membership `x %in% Y`; and they combine with `&`, `|`, `!`.

```
x <- c(2, 1, 3, 0)
x == 0 | x == 1
```

```
## [1] FALSE  TRUE FALSE  TRUE
```

```
x %in% c(0, 1)
```

```
## [1] FALSE  TRUE FALSE  TRUE
```

```
filter(presidential, party == "Republican")
```

```
## # A tibble: 7 x 4
##   name       start      end        party
##   <chr>      <date>     <date>     <chr>
## 1 Eisenhower 1953-01-20 1961-01-20 Republican
## 2 Nixon      1969-01-20 1974-08-09 Republican
## 3 Ford       1974-08-09 1977-01-20 Republican
## 4 Reagan     1981-01-20 1989-01-20 Republican
```

```
## 5 Bush       1989-01-20 1993-01-20 Republican
## # i 2 more rows
```

Missing values are contagious: `NA > 5`, `NA + 10` and even `NA == NA` are `NA`. Test with `is.na()`; to drop observations with a missing variable, `filter(!is.na(variable))`.

select() takes the data and a list of columns. For wide data the helpers `starts_with()`, `ends_with()`, `contains()`, `num_range()` select by name pattern:

```
flights %>% select(ends_with("time"))
```

```
## # A tibble: 336,776 x 5
##   dep_time sched_dep_time arr_time sched_arr_time air_time
##      <int>          <int>    <int>          <int>    <dbl>
## 1      517            515      830            819      227
## 2      533            529      850            830      227
## 3      542            540      923            850      160
## 4      544            545     1004           1022      183
## 5      554            600      812            837      116
## # i 336,771 more rows
```

Combining verbs and the pipe. Which Democratic presidents served since Watergate? Nesting `select(filter(presidential, start > 1973 & party == "Democratic"), name)` works but reads inside out. The pipe `%>%` (keyboard shortcut Ctrl+Shift+M) passes the left side as the first argument, so `df %>% filter(cond)` equals `filter(df, cond)`:

```
presidential %>%
  filter(start > 1973 & party == "Democratic") %>%
  select(name)
```

```
## # A tibble: 3 x 1
##   name
##   <chr>
## 1 Carter
## 2 Clinton
## 3 Obama
```

Read it as "take `presidential`, then keep the Democrats who started after 1973, then select the name".

mutate() creates variables. The length of each term in years, using lubridate's intervals and durations:

```
my_presidents <- presidential %>%
  mutate(term_length = interval(start, end) / dyears(1))
```

It is good style to save the result as a new object rather than overwrite data from an external source.

arrange() sorts a data frame (`sort()` sorts only vectors); ties in the first key are broken by later keys:

```
my_presidents %>% arrange(desc(term_length), party)
```

```
## # A tibble: 12 x 5
##   name       start      end        party      term_length
##   <chr>      <date>     <date>     <chr>            <dbl>
## 1 Clinton    1993-01-20 2001-01-20 Democratic           8
## 2 Obama      2009-01-20 2017-01-20 Democratic           8
## 3 Eisenhower 1953-01-20 1961-01-20 Republican           8
## 4 Reagan     1981-01-20 1989-01-20 Republican           8
## 5 Bush       2001-01-20 2009-01-20 Republican           8
## # i 7 more rows
```

summarize() and group_by(). Alone, `summarize()` reduces each column to one value; `n()` counts rows and `sum(party == "Democratic")` counts the rows satisfying a condition, since `TRUE` counts as 1.

```
my_presidents %>%
  summarize(N = n(), first_year = min(year(start)), last_year = max(year(end)),
            num_dems = sum(party == "Democratic"),
            years = sum(term_length), avg_term_length = mean(term_length))
```

```
## # A tibble: 1 x 6
##       N first_year last_year num_dems years avg_term_length
##   <int>      <dbl>     <dbl>    <int> <dbl>           <dbl>
## 1    12       1953      2021        5    68            5.67
```

Units [Supplement]

The slides compute `years = sum(term_length) / 365.25` and obtain 0.186. Since `term_length` is already measured in years, the division is not needed: the twelve presidents together served about 68 years (1953–2021).

Grouping makes the same computations run for each party separately, which turns the summary into a comparison:

```
my_presidents %>%
  group_by(party) %>%
  summarize(N = n(), avg_term_length = mean(term_length),
            std_term_length = sd(term_length))
```

```
## # A tibble: 2 x 4
##   party          N avg_term_length std_term_length
##   <chr>      <int>           <dbl>           <dbl>
## 1 Democratic     5            5.6             2.34
## 2 Republican     7            5.71            2.32
```

Average terms hardly differ between the parties.

3.2 Relational data

Motivation. A database stores different kinds of things in different tables and links them by keys, both to save space and to keep each fact in one place. Analyses then need to bring the tables back together.

Multiple linked tables are called **relational data**, because the relations, not only the tables, matter. dplyr has three families of two-table verbs: *mutating joins* (add variables from matching rows of another table), *filtering joins* (keep or drop rows depending on whether they match) and *set operations* (treat rows as set elements). All assume tidy tables.

In `nycflights13`, storing "American Airlines Inc." in each of 336 776 rows of `flights` would waste space and invite inconsistencies, so `flights` stores the two-letter code and `airlines` maps codes to names. The keys: `flights` connects to `airlines` by `carrier`, to `planes` by `tailnum`, to `airports` by `origin` and `dest`, and to `weather` by `origin` and `year`, `month`, `day`, `hour`.

inner_join() keeps rows with a match in both tables:

```
flights_joined <- flights %>%
  inner_join(airlines, by = c("carrier" = "carrier"))
flights_joined %>% select(carrier, name, flight, origin, dest)
```

```
## # A tibble: 336,776 x 5
##   carrier name                  flight origin dest
##   <chr>   <chr>                  <int> <chr>  <chr>
## 1 UA      United Air Lines Inc.   1545 EWR    IAH
## 2 UA      United Air Lines Inc.   1714 LGA    IAH
## 3 AA      American Airlines Inc.  1141 JFK    MIA
## 4 B6      JetBlue Airways          725 JFK    BQN
```

```
## 5 DL      Delta Air Lines Inc.      461 LGA    ATL
## # i 336,771 more rows
```

left_join() keeps all rows of the first table, filling `NA` where there is no match. Restrict airports to the Pacific time zone and compare:

```
airports_pt <- airports %>% filter(tz == -8)
flights %>% inner_join(airports_pt, by = c("dest" = "faa")) %>% nrow()
```

```
## [1] 46324
```

```
flights %>%
  left_join(airports_pt, by = c("dest" = "faa")) %>%
  summarize(num_flights = n(),
            num_flights_pt = sum(!is.na(name)),
            num_flights_not_pt = sum(is.na(name)))
```

```
## # A tibble: 1 x 3
##   num_flights num_flights_pt num_flights_not_pt
##         <int>          <int>              <int>
## 1      336776          46324             290452
```

The inner join keeps only the 46 324 west-coast flights; the left join keeps all flights and marks the others with a missing airport name. Other joins: `right_join()` keeps all rows of the second table, `full_join()` all rows of both, `semi_join()` the rows of the first table that have a match (without adding columns), and `anti_join()` those that do not.

3.3 Example: Uber pickups in New York

Motivation. A realistic wrangling task: several monthly files, date-times stored as text, and questions about when and where demand occurs. The steps are always the same: combine, parse, derive variables, group, summarise, plot.

The data are about 4.5 million Uber pickups in New York City, April–September 2014, in six monthly CSV files with columns `Date/Time`, `Lat`, `Lon` and `Base` (the company code). The files are not in an R package, so the code is shown without output.

```
files <- sprintf("data/uber-raw-data-%s14.csv", c("apr", "may", "jun", "jul", "aug", "
    sep"))
data_2014 <- map_dfr(files, read_csv) %>%          # stack the six months
  mutate(dt = mdy_hms(`Date/Time`),
         month = month(dt, label = TRUE), day = day(dt),
         dayofweek = wday(dt, label = TRUE), hour = hour(dt))
```

The slides combine the files with `rbind()` and parse with `as.POSIXct()`; `map_dfr()` and lubridate do the same in fewer steps [Supplement]. Each question is then a grouped count followed by a bar chart:

```
data_2014 %>%
  count(hour, name = "Total") %>%
  ggplot(aes(hour, Total)) +
  geom_col(fill = "steelblue") +
  scale_y_continuous(labels = scales::comma) +
  ggtitle("Trips every hour")
data_2014 %>%
  count(month, dayofweek, name = "Total") %>%
  ggplot(aes(month, Total, fill = dayofweek)) +
  geom_col(position = "dodge")
```

Trips by hour, by hour and month, by day of the month, by month and by weekday within month all

follow this pattern; `scales::comma` formats large counts. Plotting `Lon` against `Lat` for all pickups, with axis limits around the city (latitude 40.58–40.92, longitude −74.15 to −73.70), draws a map of New York from the data alone, and colouring by `Base` shows the areas served by each company. A small lookup table of base codes and names can be joined with `left_join(by = c("Base" = "base_code"))` to label them.

Overplotting millions of points [Supplement]

With 4.5 million points, `geom_point()` is slow and saturates. Smaller points with transparency (`size = 0.1, alpha = 0.05`), or 2-d bins (`geom_bin2d()`, `geom_hex()`), show density instead of a solid blob.

Lecture 4. Statistical Foundations

4.1 From description to statistical methods

Motivation. Wrangling and visualisation describe the data in hand. To say anything beyond those particular cases, about the next flight or the whole population, we need to know how much the numbers would change with different data. That is the job of statistical methods.

The objective of data science is to extract meaningful information from data. Wrangling makes data easier to interpret and visualisation connects our minds with the data; statistical methods quantify patterns and their strength, find patterns too complex to see, and interpret data, usually through models.

Exploratory data analysis checks data integrity and generates hypotheses ("smoking is related to cancer", "shape and size predict price"), iterating transform–visualise–model. **Confirmatory** analysis checks that a fitted model is appropriate and that a hypothesis holds, either for prediction accuracy or for statistical inference. As a rule, data used for exploration are not used for confirmation.

A statistical method is more than a formula. The t-test for $H_0 : \mu = 0$ rejects when $|t| > 1.96$, $t = \sqrt{n}\,\bar{X}/s$; behind it lie

- a **model**: observations centred at an unknown μ with random error;
- a **fit**: centre and error estimated by $\bar{X}$ and s;
- **assumptions**: numeric, (approximately) normal observations;
- **uncertainty quantification**: the 95% confidence interval $\bar{X} \pm 1.96\, s/\sqrt{n}$;
- **strength of evidence**: the smaller the p-value, the stronger the evidence against the hypothesis.

The key to using such methods correctly is to view the data as a **sample** from a larger **population**, and to imagine the other samples that might have been drawn.

4.2 Samples and populations: a travel policy

Motivation. A concrete decision makes the abstract question precise: how early must a traveller arrive to be late only rarely? Pretending to know the whole population lets us see exactly how much a small sample can mislead.

A business traveller from New York has a meeting in San Francisco at a given time. How much earlier should the scheduled arrival be so that a flight delay makes the traveller late only rarely? Pretend that the 2013 flights to SFO are the whole population:

```
SF <- flights %>% filter(dest == "SFO", !is.na(arr_delay))
set.seed(101)
sf_25 <- SF %>% slice_sample(n = 25)
sf_25 %>% fivenum_tbl(arr_delay)
```

```
## # A tibble: 1 x 8
##       n  mean    sd   min   q25 median   q75   max
##   <int> <dbl> <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1    25  0.12  29.8   -38   -23     -5    14   103
```

```
SF %>% fivenum_tbl(arr_delay)
```

```
## # A tibble: 1 x 8
##       n  mean    sd   min   q25 median   q75   max
##   <int> <dbl> <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1 13173  2.67  47.7   -86   -23     -8    12  1007
```

`slice_sample()` draws random rows and `set.seed()` makes the pseudo-random draw reproducible. The sample's largest delay is under two hours, but the population contains a delay of 1007 minutes, about 17 hours. Planning for the worst case is absurd; a sensible policy accepts a small risk, say being late 2% of the time, i.e. uses the 98th percentile:

```
sf_25 %>% summarize(q98 = quantile(arr_delay, p = 0.98))
```

```
## # A tibble: 1 x 1
##     q98
##   <dbl>
## 1  67.5
```

```
SF %>% group_by(arr_delay < 90) %>% count() %>% mutate(pct = n / nrow(SF))
```

```
## # A tibble: 2 x 3
## # Groups:   arr_delay < 90 [2]
##   `arr_delay < 90`     n    pct
##   <lgl>            <int>  <dbl>
## 1 FALSE              640 0.0486
## 2 TRUE             12533 0.951
```

```
SF %>% summarize(q98 = quantile(arr_delay, p = 0.98))
```

```
## # A tibble: 1 x 1
##     q98
##   <dbl>
## 1   153
```

A 90-minute policy would fail about 5% of the time, not 2%; the population says about 150 minutes. Computing a percentile is easy; the real question is how good an estimate from a sample is, and how large a sample must be. That is where statistics comes in.

4.3 Sampling distributions

Motivation. A sample statistic is itself random: another sample would give another value. Its sampling distribution, and above all the standard error, measures how reliable it is.

A **statistic** is a number summarising data (mean, standard deviation, median, maximum). Extremes such as the maximum are not useful statistics, because there is no reliable way to tell how well they reflect the population.

How similar would the statistic be on a new sample? With the population at hand we can just draw many samples:

```
n <- 25
num_trials <- 500
sf_25_means <- 1:num_trials %>%
  map_dfr(~ SF %>% slice_sample(n = n) %>% summarize(mean_arr_delay = mean(arr_delay)))
    %>%
  mutate(n = n)
sf_25_means %>% fivenum_tbl(mean_arr_delay)
```

```
## # A tibble: 1 x 8
##       n  mean    sd   min   q25 median   q75   max
##   <int> <dbl> <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1   500  3.09  10.2 -20.1 -3.72   1.54     9  56.7
```

Vocabulary:

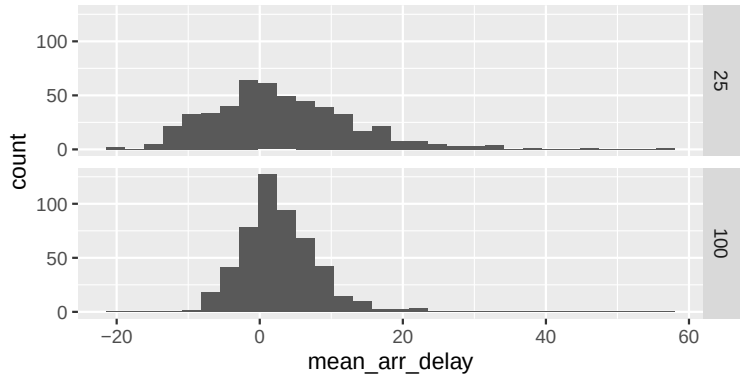
- the **sample size** n;
- the **sampling distribution**: the collection of the statistic over many trials (the number of trials is not important once large);
- its **shape**: here somewhat right-skewed;
- the **standard error**: the standard deviation of the sampling distribution, i.e. its width;
- the **95% confidence interval**, computed from the mean and standard error of the sampling distribution, roughly mean ± 2 SE.

For $n = 25$ the sample means range over about 75 minutes, which shows how unreliable such a small sample is. Repeating with $n = 100$:

```
sf_100_means <- 1:num_trials %>%
  map_dfr(~ SF %>% slice_sample(n = 100) %>% summarize(mean_arr_delay = mean(arr_delay))
    ) %>%
  mutate(n = 100)
bind_rows(sf_25_means, sf_100_means) %>%
  group_by(n) %>%
  summarize(mean = mean(mean_arr_delay), se = sd(mean_arr_delay))
```

```
## # A tibble: 2 x 3
##       n  mean    se
##   <dbl> <dbl> <dbl>
## 1    25  3.09 10.2
## 2   100  2.58  4.99
```

```
bind_rows(sf_25_means, sf_100_means) %>%
  ggplot(aes(mean_arr_delay)) + geom_histogram(bins = 30) + facet_grid(n ~ .)
```



Quadrupling n halves the standard error and makes the distribution more symmetric.

The $1/\sqrt{n}$ law [Supplement]

For a sample mean of i.i.d. observations, $\mathrm{Var}(\bar{X}) = \sigma^2/n$, so $\mathrm{SE} = \sigma/\sqrt{n}$: here $\sigma \approx 47.7$ gives 9.5 for $n = 25$ and 4.8 for $n = 100$, matching the simulation. (Sampling without replacement from a finite population multiplies by $\sqrt{(N-n)/(N-1)} \approx 1$ here.) By the central limit theorem the sampling distribution approaches a normal shape as n grows, even for skewed data such as delays.

4.4 The bootstrap

Motivation. In practice we have one sample, not the population, so we cannot draw new samples. The bootstrap pretends that the sample is the population and resamples from it, which approximates the sampling distribution from the data alone.

Resampling draws a new sample from the existing sample. When sampling from a population no case is duplicated; when resampling we sample *with replacement*, and the duplication is what produces variability. Take one sample of $n = 200$ and forget the population:

```
n <- 200
orig_sample <- SF %>% slice_sample(n = n, replace = FALSE)
sf_200_bs <- 1:num_trials %>%
  map_dfr(~ orig_sample %>% slice_sample(n = n, replace = TRUE) %>%
            summarize(mean_arr_delay = mean(arr_delay)))
sf_200_bs %>% fivenum_tbl(mean_arr_delay)
```

```
## # A tibble: 1 x 8
##       n  mean    sd   min   q25 median   q75   max
##   <int> <dbl> <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1   500  8.58  4.16 -1.94  5.47   8.42  11.3  23.7
```

```
sf_200_pop <- 1:num_trials %>%
  map_dfr(~ SF %>% slice_sample(n = n) %>% summarize(mean_arr_delay = mean(arr_delay)))
sf_200_pop %>% fivenum_tbl(mean_arr_delay)
```

```
## # A tibble: 1 x 8
##       n  mean    sd   min   q25 median   q75   max
##   <int> <dbl> <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1   500  2.69  3.32 -6.41 0.391   2.65  4.86  18.0
```

The **bootstrap distribution** (first table) is centred at the original sample's mean rather than the population mean, but its standard deviation approximates the true standard error (second table). For moderate to large samples and enough resamples it has been proven to approximate the aspects of the sampling distribution we care about most, such as standard errors and quantiles (Efron and Tibshirani, 1993).

Back to the travel policy: the 98th percentile from the sample, and its bootstrap reliability:

```
orig_sample %>% summarize(q98 = quantile(arr_delay, p = 0.98))
```

```
## # A tibble: 1 x 1
##     q98
##   <dbl>
## 1  209.
```

```
sf_200_bs_q98 <- 1:num_trials %>%
  map_dfr(~ orig_sample %>% slice_sample(n = n, replace = TRUE) %>%
            summarize(q98 = quantile(arr_delay, p = 0.98)))
sf_200_bs_q98 %>% fivenum_tbl(q98)
```

```
## # A tibble: 1 x 8
##       n  mean    sd   min   q25 median   q75   max
##   <int> <dbl> <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl>
## 1   500  201.  58.2  74.1  152.    208  251.   360
```

The standard error of the estimated 98th percentile is large relative to the estimate: extreme quantiles need much more data than means.

4.5 Outliers

Motivation. Rare extreme events are exactly what a travel policy must anticipate. Looking at them can suggest which conditions make long delays likely.

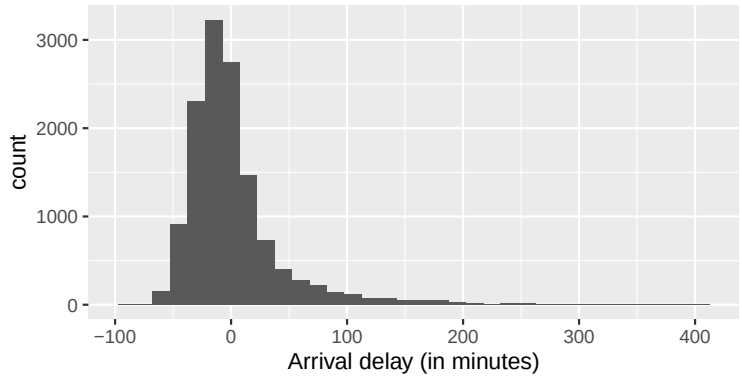
Consider delays of at least 7 hours (420 minutes) as extreme:

```
SF %>% filter(arr_delay >= 420) %>% select(month, day, dep_delay, arr_delay, carrier)
```

```
## # A tibble: 7 x 5
##   month   day dep_delay arr_delay carrier
##   <int> <int>     <dbl>     <dbl> <chr>
## 1    12     7       374       422 UA
## 2     7     6       589       561 DL
## 3     7     7       629       676 VX
## 4     7     7       653       632 VX
## 5     7    10       453       445 B6
## 6     7    10       432       433 VX
## # i 1 more row
```

Removing them shows the main part of the distribution: most flights arrive on time or less than an hour late.

```
SF %>% filter(arr_delay < 420) %>%
  ggplot(aes(arr_delay)) + geom_histogram(binwidth = 15) +
  labs(x = "Arrival delay (in minutes)")
```



The outliers suggest that month or carrier matter. Counting long delays (over an hour) by month and by carrier:

```
SF %>% mutate(long_delay = arr_delay > 60) %>%
  count(month, long_delay) %>%
  pivot_wider(names_from = month, values_from = n)
```

```
## # A tibble: 2 x 13
##   long_delay   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`  `10`  `11`  `12`
##   <lgl>      <int> <int> <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1 FALSE        856   741   812   993  1128   980   966  1159  1124  1177  1107  1093
## 2 TRUE          29    21    61   112    65   209   226    96    65    36    51    66
```

```
SF %>% mutate(long_delay = arr_delay > 60) %>%
  count(carrier, long_delay) %>%
  pivot_wider(names_from = carrier, values_from = n)
```

```
## # A tibble: 2 x 6
##   long_delay    AA    B6    DL    UA    VX
##   <lgl>      <int> <int> <int> <int> <int>
## 1 FALSE       1250   934  1757  6236  1959
## 2 TRUE         148    86    91   492   220
```

June and July are problem months, and Delta (DL) performs reasonably. This hints at a policy: arrive extra early in June and July, and consider Delta for travel to SFO.

4.6 Statistical models

Motivation. Conditioning on month and carrier explained part of the variation in delays. A statistical model does this systematically, relating the response to several variables at once and separating signal from noise.

"Essentially, all models are wrong, but some are useful" (G. E. P. Box). A model is an idealised representation of a system, like a weather forecast, which is useful though often wrong. We build models

1. to **understand** complex phenomena (which factors matter?), preferring simple interpretable models; and
2. to **predict** unseen data accurately, sometimes with uninterpretable black-box models.

Most of the time we balance interpretability and accuracy.

With response Y and predictors $X = (X_1, \dots, X_p)$, we assume

$$Y = f(X) + \epsilon,$$

with f fixed but unknown and ϵ random error with mean zero. We estimate f for **prediction**, $\hat{Y} = \hat{f}(X)$, where accuracy matters, or for **inference**, where the form of f matters: which predictors are associated with the response, how, and whether a linear form suffices.

Proposition 4.1 (reducible and irreducible error)

If $\hat{f}$ and X are fixed and $\mathbb{E}\epsilon = 0$, then

$$\mathbb{E}[(Y - \hat{Y})^2] = [f(X) - \hat{f}(X)]^2 + \mathrm{Var}(\epsilon).$$

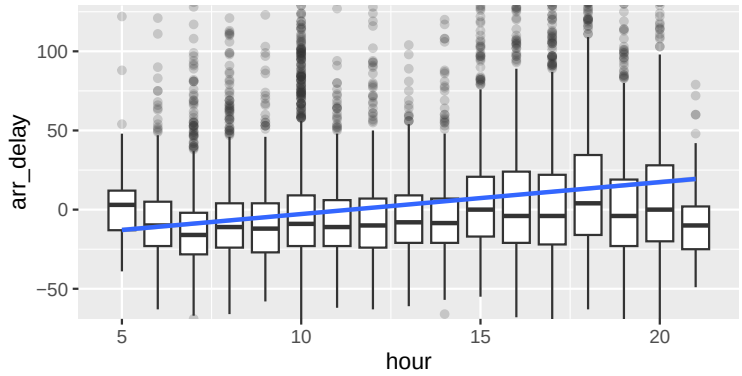
Proof [Supplement]. $Y - \hat{Y} = \{f(X) - \hat{f}(X)\} + \epsilon$, where the first term is a constant c. Hence $\mathbb{E}(Y - \hat{Y})^2 = c^2 + 2c\,\mathbb{E}\epsilon + \mathbb{E}\epsilon^2 = c^2 + \mathrm{Var}(\epsilon)$. ■

The first term, the **reducible error**, can be lowered by a better estimate $\hat{f}$; the second, the **irreducible error**, cannot. Methods aim to minimise the reducible error.

Parametric models specify $f(X;\theta)$ with finitely many parameters, e.g. $Y \approx \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p$; **nonparametric** models such as k-nearest neighbours make no explicit assumption about the form of f. More flexible methods are generally less interpretable.

Example. Does the scheduled hour of departure affect delays, as the theory that delays cascade through the day predicts? Flights to SFO are scheduled from 5 am to 9 pm. A linear model tracks the mean delay over the day:

```
SF %>% ggplot(aes(hour, arr_delay)) +
  geom_boxplot(aes(group = hour), outlier.alpha = 0.2) +
  geom_smooth(method = "lm", formula = y ~ x) +
  coord_cartesian(ylim = c(-60, 120))
```




```
mod1 <- lm(arr_delay ~ hour, data = SF)
broom::tidy(mod1)
```

```
## # A tibble: 2 x 5
##   term        estimate std.error statistic   p.value
##   <chr>          <dbl>     <dbl>     <dbl>     <dbl>
## 1 (Intercept)   -22.9     1.23       -18.6 2.88e- 76
## 2 hour            2.01    0.0915      22.0 1.78e-105
```

The mean arrival delay increases by about 2 minutes for each hour later in the day, and the tiny p-value says such a slope would be very unlikely if there were no relationship.

Lecture 5. Linear Regression

5.1 Simple linear regression

Motivation. The most common question in statistics is whether a variable Y is associated with predictors $X_1, \ldots, X_p$, how, and whether the association can be used to predict Y. Linear regression is the simplest complete answer, and its ideas (a loss, a fitted family, uncertainty of the fit) carry over to every later model.

Two related tasks: **estimation** uses data to determine model parameters; **prediction** uses the fitted model to predict outputs for unseen data. We call y the *outcome* (response) and x the *predictor* (explanatory variable), avoiding "dependent" and "independent", which have other meanings in statistics.

The simple linear regression model is

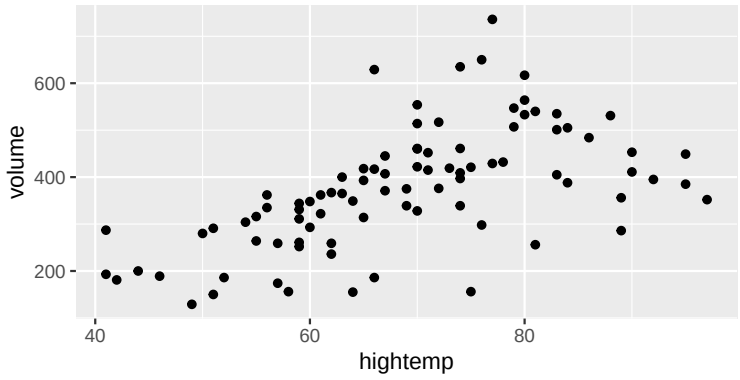
$$y_i = \beta_0 + \beta_1 x_i + \epsilon_i, \qquad i = 1, \ldots, n,$$

with population intercept β_0, slope β_1 and random errors ϵ_i with mean 0. The coefficients are unknown; `lm()` finds the best estimates $\hat{\beta}_0$, $\hat{\beta}_1$, giving fitted values $\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i$ and residuals $\hat{\epsilon}_i = y_i - \hat{y}_i$.

5.1.1 Example: the rail trail

`mosaicData::RailTrail` records daily ridership (`volume`, riders and walkers on a bike path in Florence, Massachusetts) over 90 days, with temperature, rainfall, cloud cover and day of the week. The planning commission wants to understand how ridership relates to these variables.

```
ggplot(RailTrail, aes(x = hightemp, y = volume)) + geom_point()
```




```
mod <- lm(volume ~ hightemp, data = RailTrail)
tidy(mod)
```

```
## # A tibble: 2 x 5
##   term        estimate std.error statistic       p.value
##   <chr>          <dbl>     <dbl>     <dbl>         <dbl>
## 1 (Intercept)   -17.1     59.4      -0.288 0.774
## 2 hightemp        5.70     0.848     6.72  0.00000000171
```

$\hat{\beta}_1 = 5.70$: about 5.7 more trail users are predicted for each additional degree (Fahrenheit) of high temperature. The intercept, the predicted volume at 0°F, is far outside the data and not interesting. `summary(mod)` adds the residual standard error, R^2 and the overall F-test:

```
summary(mod)
```

```
##
## Call:
## lm(formula = volume ~ hightemp, data = RailTrail)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -254.562  -57.800    8.737   57.352  314.035
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -17.079     59.395  -0.288    0.774
## hightemp       5.702      0.848   6.724 1.71e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 104.2 on 88 degrees of freedom
## Multiple R-squared:  0.3394, Adjusted R-squared:  0.3319
## F-statistic: 45.21 on 1 and 88 DF,  p-value: 1.705e-09
```

5.1.2 Prediction and intervals

```
predict(mod, newdata = data.frame(hightemp = c(30, 40)))
```

```
##        1        2
## 153.9771 210.9958
```

```
predict(mod, interval = "confidence")[1:3, ]
```

```
##        fit      lwr      upr
## 1 456.1766 423.8301 488.5231
## 2 399.1578 376.2313 422.0843
## 3 404.8597 381.3622 428.3572
```

```
predict(mod, interval = "prediction")[1:3, ]
```

```
## Warning in predict.lm(mod, interval = "prediction"): predictions on current data refer to
    _future_ responses
```

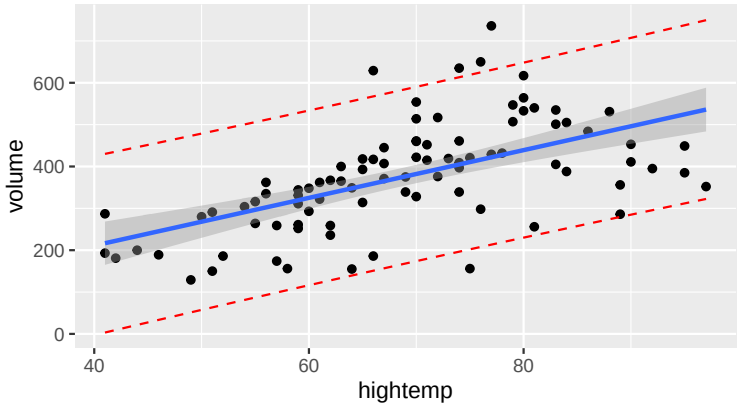
```
##        fit      lwr      upr
## 1 456.1766 246.6175 665.7357
## 2 399.1578 190.8447 607.4709
## 3 404.8597 196.4830 613.2364
```

The **confidence interval** is for the mean volume at that temperature; the **prediction interval** is for one new day, and is much wider because it adds the day-to-day noise ϵ.

```
pi <- predict(mod, interval = "prediction")
```

```
## Warning in predict.lm(mod, interval = "prediction"): predictions on current data refer to
    _future_ responses
```

```
RailTrail %>%
  mutate(lower.pi = pi[, "lwr"], upper.pi = pi[, "upr"]) %>%
  ggplot(aes(x = hightemp, y = volume)) +
  geom_point() +
  geom_smooth(method = "lm", formula = y ~ x, se = TRUE) +
  geom_line(aes(y = lower.pi), colour = "red", linetype = "dashed") +
  geom_line(aes(y = upper.pi), colour = "red", linetype = "dashed")
```



The two interval formulas [Supplement]

With $S_{xx} = \sum(x_i - \bar{x})^2$ and $s^2 = \text{SSE}/(n-2)$, the intervals at x_0 are

$$\hat{y}_0 \pm t_{n-2,0.975}\, s\sqrt{\frac{1}{n} + \frac{(x_0-\bar{x})^2}{S_{xx}}} \quad \text{(mean)}, \qquad \hat{y}_0 \pm t_{n-2,0.975}\, s\sqrt{1 + \frac{1}{n} + \frac{(x_0-\bar{x})^2}{S_{xx}}} \quad \text{(new obs.)}.$$

The extra 1 under the second root is the variance of the new error; both widen away from $\bar{x}$.

5.1.3 Strength of fit and least squares

With two variables the correlation r measures the strength of the linear relationship; in general we use

$$R^2 = 1 - \frac{\text{SSE}}{\text{SST}} = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \in [0,1],$$

the fraction of variation explained, comparing the fitted model with the null model $y_i = \beta_0 + \epsilon_i$. Since SST is fixed, a smaller SSE is a better fit, and least squares finds the line with the smallest SSE:

$$\sum_i \{y_i - (\hat{\beta}_0 + \hat{\beta}_1 x_i)\}^2 \le \sum_i \{y_i - (\beta_0 + \beta_1 x_i)\}^2 \quad \text{for all } \beta_0, \beta_1.$$

Least squares became standard partly for computational convenience, which still matters for big data; like the mean, it is sensitive to outliers, mainly a problem in small or moderate samples. More generally, *any statistical method optimises a goodness-of-fit measure over a family of models*: sometimes the optimum has a formula, often it must be approximated by an algorithm, and comparing several families makes computational efficiency important.

5.1.4 A categorical predictor

With the indicator `weekday` (1 on weekdays, 0 on weekends and holidays) the model $\text{volume}_i = \beta_0 + \beta_1 \text{weekday}_i + \epsilon_i$ has only two fitted values, the two group means:

```
RailTrail %>% group_by(weekday) %>% summarise(mean_volume = mean(volume))
```

```
## # A tibble: 2 x 2
##   weekday mean_volume
##   <lgl>         <dbl>
## 1 FALSE          431.
## 2 TRUE           350.
```

```
coef(lm(volume ~ weekday, data = RailTrail))
```

```
## (Intercept) weekdayTRUE
##   430.71429   -80.29493
```

About 80 fewer riders are expected on a weekday. `lm()` takes the alphabetically first level as the **reference group** and creates indicators for the others; recoding changes the reference and flips the sign:

```
RailTrail <- RailTrail %>%
  mutate(day = ifelse(weekday == 1, "weekday", "weekend/holiday"))
coef(lm(volume ~ day, data = RailTrail))
```

```
##        (Intercept) dayweekend/holiday
##          350.41935           80.29493
```

5.2 Loss functions

Motivation. "Best fit" needs a definition. Choosing the loss function defines it, and different losses give different answers, most visibly when the data contain outliers.

A **loss function** $L(y, \hat{y})$ measures the cost of predicting $\hat{y}$ when the truth is y. The two most common are the squared (L_2) loss $(y-\hat{y})^2$ and the absolute (L_1) loss $|y-\hat{y}|$. Both are large when $\hat{y}$ is far from y, but L_2 penalises large residuals much more.

The model is fitted by minimising the average loss over the data, the **empirical risk**

$$\hat{R}(\beta_0, \beta_1) = \frac{1}{n}\sum_i L(y_i, \hat{y}_i),$$

which measures fit to the sample, not the population. With L_2 it is the mean squared error (MSE), and with L_1 the mean absolute error (MAE). Least squares is exactly minimisation of the L_2 risk. Setting the derivatives to zero gives the estimating equations $\frac{1}{n}\sum_i (y_i - \hat{y}_i) = 0$ and $\frac{1}{n}\sum_i (y_i - \hat{y}_i)x_i = 0$, whose solution is

$$\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \qquad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}.$$

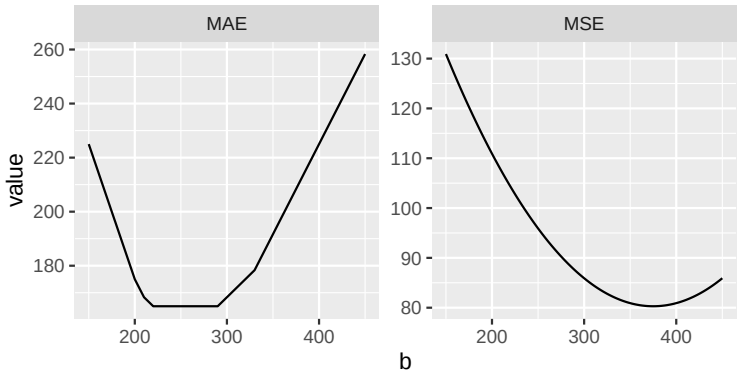
Proposition 5.1 (constant model)

For the constant model $y = \beta_0 + \epsilon$, the minimiser of the L_2 risk is $\bar{y}$ and a minimiser of the L_1 risk is any median of $y_1, \dots, y_n$.

Proof. L_2: $\frac{d}{d\beta_0}\frac{1}{n}\sum(y_i-\beta_0)^2 = -\frac{2}{n}\sum(y_i-\beta_0) = 0$ gives $\beta_0 = \bar{y}$, and the risk is a convex parabola. L_1: away from the data points, $\frac{d}{d\beta_0}\frac{1}{n}\sum|y_i-\beta_0| = \frac{1}{n}(\#\{y_i < \beta_0\} - \#\{y_i > \beta_0\})$. The risk is convex and piecewise linear, decreasing while fewer than half the points lie below β_0 and increasing once more than half do, so it is minimised where the numbers of points below and above balance, i.e. at a median. ∎

Example (bubble-tea sales). Weekday sales on five random days are 200, 210, 220, 290, 330. A constant model predicts 250 (mean, L_2) or 220 (median, L_1). Adding an outlying day of 1000 moves the mean to 375 but the median only to 255; and with six points the L_1 minimiser is not unique: any value between 220 and 290 minimises the MAE.

```
y <- c(200, 210, 220, 290, 330, 1000)
tibble(b = seq(150, 450, by = 1)) %>%
  mutate(MSE = map_dbl(b, ~ mean((y - .x)^2)) / 1000,
         MAE = map_dbl(b, ~ mean(abs(y - .x)))) %>%
  pivot_longer(-b) %>%
  ggplot(aes(b, value)) + geom_line() + facet_wrap(~ name, scales = "free_y")
```



MSE is smooth (easy to minimise) with a unique minimum at the mean but is sensitive to outliers; MAE is piecewise linear (harder to minimise), flat between 220 and 290, and robust. (MSE is shown in thousands.)

5.3 Multiple regression

Motivation. Ridership depends on temperature, rain and the day of the week at the same time. Multiple regression estimates each effect while holding the others fixed, which is what makes its coefficients meaningful but also subtle.

The model is $y = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p + \epsilon$. Each β_j is the predicted change in y for a one-unit increase in x_j *holding the other predictors fixed.*

```
mod_plane <- lm(volume ~ hightemp + precip, data = RailTrail)
tidy(mod_plane)
```

```
## # A tibble: 3 x 5
##   term        estimate std.error statistic  p.value
##   <chr>          <dbl>     <dbl>     <dbl>    <dbl>
## 1 (Intercept)   -31.5     55.2      -0.571 5.70e- 1
## 2 hightemp        6.12     0.794     7.70  1.97e-11
## 3 precip       -153.      39.3      -3.90  1.90e- 4
```

An inch of rain lowers predicted ridership by about 153 at a given temperature. Three questions guide multiple regression: how uncertain are the coefficients, which predictors are useful, and how accurate are predictions?

5.3.1 Inference for coefficients

Uncertainty arises in predicting y from x and in estimating the coefficients. The 95% confidence interval by normal approximation is $\hat{\beta}_j \pm 1.96\,\mathrm{SE}(\hat{\beta}_j)$; `confint()` uses t quantiles:

```
mod_p_planes <- lm(volume ~ hightemp + precip + weekday, data = RailTrail)
tidy(mod_p_planes)
```

```
## # A tibble: 4 x 5
##   term        estimate std.error statistic  p.value
##   <chr>          <dbl>     <dbl>     <dbl>    <dbl>
## 1 (Intercept)    19.3     60.3       0.320 7.50e- 1
## 2 hightemp        5.80     0.799     7.26  1.59e-10
## 3 precip       -146.      38.9      -3.74  3.27e- 4
## 4 weekdayTRUE   -43.1     22.2      -1.94  5.52e- 2
```

```
confint(mod_p_planes)
```

```
##                    2.5 %      97.5 %
## (Intercept) -100.630740 139.2683959
```

```
## hightemp       4.213254   7.3880945
## precip      -222.927105 -68.2908651
## weekdayTRUE  -87.265016   0.9763902
```

Without the population we can still use the bootstrap (Lecture 4): resample the days, refit, and take the standard deviation of the coefficients.

```
set.seed(1)
n <- nrow(RailTrail)
coef_fun <- function(data) lm(volume ~ hightemp + precip + weekday, data = data)$coef
boot <- 1:500 %>%
  map_dfr(~ RailTrail %>% slice_sample(n = n, replace = TRUE) %>% coef_fun())
round(coef(mod_p_planes)["hightemp"] + c(-1.96, 1.96) * sd(boot$hightemp), 2)
```

```
## [1] 4.2 7.4
```

The bootstrap interval for `hightemp` is close to the model-based one.

5.3.2 Choosing a model

Adding variables always increases R^2, but a bigger model is not necessarily better. **Occam's razor**: other things equal, prefer the simpler model. Candidate models, from small to full (`volume ~ .` uses all variables):

```
mod1 <- lm(volume ~ hightemp + weekday, data = RailTrail)
mod2 <- lm(volume ~ hightemp + cloudcover + weekday, data = RailTrail)
mod3 <- lm(volume ~ . - day, data = RailTrail)
map_dfr(list(mod1 = mod1, mod2 = mod2, mod3 = mod3), glance, .id = "model") %>%
  select(model, r.squared, adj.r.squared, AIC, BIC)
```

```
## # A tibble: 3 x 5
##   model r.squared adj.r.squared   AIC   BIC
##   <chr>     <dbl>         <dbl> <dbl> <dbl>
## 1 mod1      0.374         0.359 1093. 1103.
## 2 mod2      0.459         0.440 1082. 1094.
## 3 mod3      0.511         0.469 1081. 1103.
```

R^2 grows with every added variable, but BIC prefers `mod2`, while adjusted R^2 and AIC slightly favour the full model: the criteria trade fit against size differently. Criteria that penalise size:

- adjusted $R^2_{\text{adj}} = 1 - (1 - R^2)\dfrac{n-1}{n-p-1}$, with p the number of predictors;
- AIC (Akaike's information criterion) and BIC (Bayesian, with a stronger penalty on p), smaller is better;
- Mallows' C_p.

Searching all 2^p subsets (**all-subset regression**) is expensive; **stepwise regression** instead drops variables from the full model (backward), adds them to the constant model (forward), or does both (`MASS::stepAIC()`). **Penalized regression** (ridge, lasso; Lecture 7) shrinks coefficients instead of removing variables.

These are in-sample methods, so model selection can **overfit** (fit the noise) and perform worse on new data; cross-validation (Lecture 6) guards against this. For linear regression overfitting is rarely severe because of the simple global structure, but for flexible, locally adaptive methods cross-validation is essential.

5.4 Other considerations

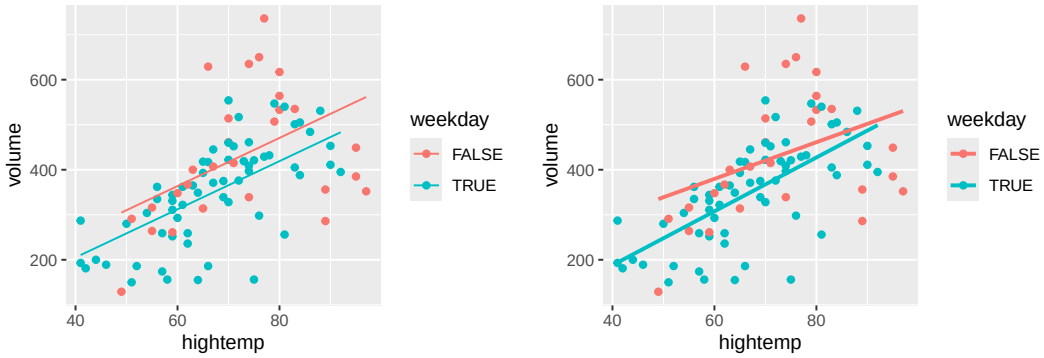
Factor variables are coded by dummy (indicator) variables: *reference coding* drops one level, *one-hot encoding* keeps all (and then drops the intercept). Ordered categories may be treated as numeric scores.

Interactions. The linear model assumes the effects are *additive* (the effect of x_1 does not depend on x_2) and *linear*. With a 0/1 variable x_2 (weekday), $\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2$ gives parallel lines: intercept $\hat{\beta}_0$ on weekends and $\hat{\beta}_0 + \hat{\beta}_2$ on weekdays, same slope $\hat{\beta}_1$. Adding $x_1 x_2$,

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \hat{\beta}_3 x_1 x_2,$$

gives slope $\hat{\beta}_1$ on weekends and $\hat{\beta}_1 + \hat{\beta}_3$ on weekdays: an **interaction model**.

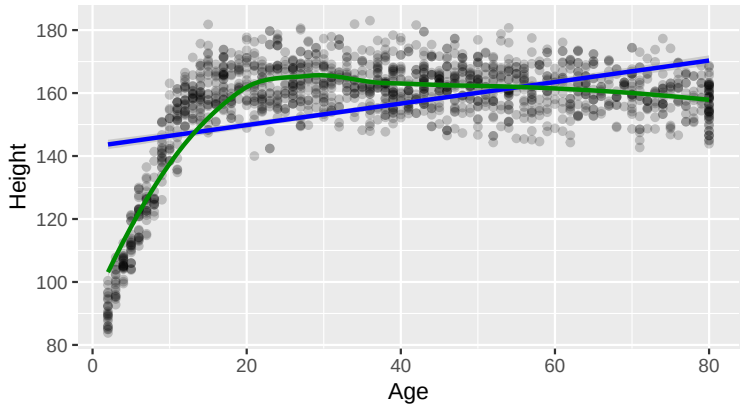
```
ggplot(RailTrail, aes(hightemp, volume, colour = weekday)) +
  geom_point() +
  geom_line(aes(y = fitted(lm(volume ~ hightemp + weekday, data = RailTrail))))
ggplot(RailTrail, aes(hightemp, volume, colour = weekday)) +
  geom_point() + geom_smooth(method = "lm", formula = y ~ x, se = FALSE)
```



Left: parallel slopes; right: separate slopes (interaction).

Nonlinear relationships. Height against age for females in `NHANES` is clearly not linear: a straight line misfits badly, while a LOESS smoother follows the growth curve. The better fit has a price: the smoother's confidence band is wider. Polynomial regression, splines, local regression and GAMs are studied in Lecture 10.

```
NHANES %>% filter(Gender == "female", !is.na(Height)) %>%
  slice_sample(n = 1500) %>%
  ggplot(aes(Age, Height)) +
  geom_point(alpha = 0.2) +
  geom_smooth(method = "lm", formula = y ~ x, colour = "blue") +
  geom_smooth(method = "loess", formula = y ~ x, colour = "green4")
```



Lecture 6. Generalized Linear Models and Model Evaluation

6.1 Generalized linear models

Motivation. Linear regression assumes a continuous outcome that can take any value. Many outcomes are binary (diabetes or not), counts (articles published), or otherwise restricted. Generalized linear models keep the linear predictor but connect it to the outcome's mean through a link function suited to its type.

So far outcomes were quantitative. For a binary outcome, an outcome with several levels, or a count, we still use a linear predictor but through a monotone **link function** f:

$$f(\theta_i) = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}, \qquad y_i \sim D(\theta_i, \psi),$$

where θ_i is the centre of the distribution D and ψ controls its scale or shape.

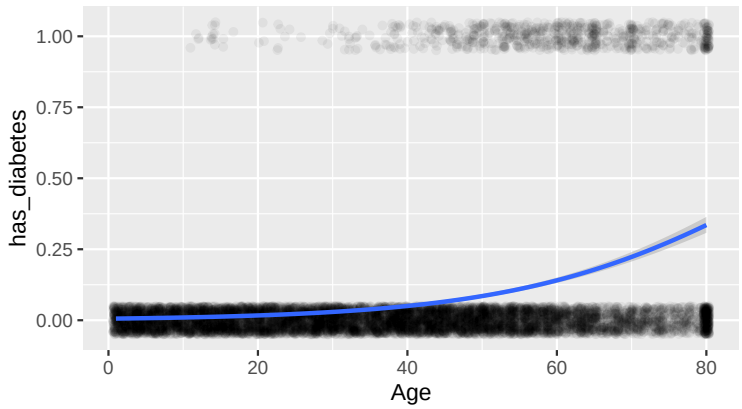
6.2 Logistic regression

For binary $y \sim \text{Bernoulli}(\theta)$ the link is the **logit**:

$$\text{logit}(\theta) = \log \frac{\theta}{1-\theta} = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p, \qquad \theta = \frac{1}{1 + \exp\{-(\beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p)\}} \in (0, 1).$$

For example, the probability of diabetes as a function of age and BMI in `NHANES`:

```
NHANES <- NHANES %>% mutate(has_diabetes = as.numeric(Diabetes == "Yes"))
NHANES %>% filter(!is.na(has_diabetes)) %>%
  ggplot(aes(Age, has_diabetes)) +
  geom_jitter(height = 0.05, alpha = 0.05) +
  geom_smooth(method = "glm", method.args = list(family = "binomial"), formula = y ~ x)
```



The probability rises with age (and similarly with BMI). Which matters more? A multiple logistic regression fits both, with `glm(..., family = "binomial")`:

```
logreg <- glm(has_diabetes ~ BMI + Age, family = "binomial", data = NHANES)
tidy(logreg)
```

```
## # A tibble: 3 x 5
##   term        estimate std.error statistic   p.value
##   <chr>          <dbl>     <dbl>     <dbl>     <dbl>
## 1 (Intercept)  -8.08     0.244       -33.1 1.30e-239
## 2 BMI           0.0943   0.00552      17.1 1.74e- 65
## 3 Age           0.0573   0.00249      23.0 2.28e-117
```

Both p-values are tiny. The predicted probability for a 60-year-old with BMI 25:

```
linear_component <- c(1, 25, 60) %*% coef(logreg)
exp(linear_component) / (1 + exp(linear_component))
```

```
##             [,1]
## [1,] 0.09233095
```

```
predict(logreg, newdata = data.frame(BMI = 25, Age = 60), type = "response")
```

```
##          1
## 0.09233095
```

6.2.1 Coefficients as odds ratios

Because the curve is nonlinear, a unit change in x does not change the probability by a constant amount. Coefficients are interpreted through **odds**, $p/(1-p)$, and **odds ratios**: if the odds of getting a job are $p/(1-p)$ with an MBA and $q/(1-q)$ without, their ratio is the factor by which an MBA changes the odds. With a single 0/1 predictor, $\operatorname{logit} P(y_i = 1) = \beta_0 + \beta_1 x_i$, the log odds are $\beta_0 + \beta_1$ when $x_i = 1$ and β_0 when $x_i = 0$, so with $p = P(y = 1 \mid x = 1)$ and $q = P(y = 1 \mid x = 0)$,

$$e^{\beta_1} = \frac{p/(1-p)}{q/(1-q)}.$$

In general e^{β_j} is the factor by which the odds are multiplied when x_j increases by one, other predictors fixed:

```
exp(coef(logreg))
```

```
##  (Intercept)          BMI          Age
## 0.0003095805 1.0989173575 1.0589497173
```

Each additional year of age multiplies the odds of diabetes by about 1.06, and each BMI unit by about 1.10.

6.3 GLMs for counts

Motivation. Counts are non-negative integers whose variance usually grows with the mean. The Poisson model captures this in the simplest way; when the data are more variable or have extra zeros, the negative binomial and zero-inflated models extend it.

Count data record how many times something happened (car accidents per day, doctor visits per year). We consider Poisson, negative binomial and zero-inflated Poisson regression.

6.3.1 Poisson regression

A Poisson variable with rate λ has $P(x = k \mid \lambda) = e^{-\lambda}\lambda^k/k!$ for $k = 0, 1, 2, \ldots$, with mean and variance both λ; it models the number of events in a fixed interval (emails per hour). Poisson regression assumes $y_i \sim \text{Poisson}(\lambda_i)$ with the log link,

$$\log \lambda_i = \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip},$$

and no extra parameter ψ. e^{β_j} is the multiplicative effect of a one-unit increase in x_j on the rate: $\lambda(x_j + 1) = \lambda(x_j)\, e^{\beta_j}$.

Example. `pscl::bioChemists` records, for 915 biochemistry PhD students, the number of articles in the last three years of the PhD (`art`), gender, marital status, number of children under 5, departmental prestige and the mentor's article count.

```
bioChemists <- bioChemists %>% mutate(gender = ifelse(fem == "Men", 1, 0))
pois_mod <- glm(art ~ gender, family = "poisson", data = bioChemists)
round(coef(pois_mod), 3)
```

```
## (Intercept)      gender
##       0.385       0.247
```

```
exp(coef(pois_mod))
```

```
## (Intercept)      gender
##    1.470309    1.280405
```

For women ($x = 0$) $\hat{\lambda} = e^{0.385} = 1.47$ articles; for men $\hat{\lambda} = e^{0.385+0.247} = 1.88$. Men publish on average $e^{0.247} = 1.28$ times as many articles. (The slides write $\lambda_{\text{women}} = e^{0.247}$; the women's rate is $e^{\hat{\beta}_0} = e^{0.385}$, which is the value 1.470 given there [Supplement].)

6.3.2 Overdispersion and the negative binomial

If the variance exceeds what the model implies, the data are **overdispersed**. For counts this means the variance is much larger than the mean; fitting a Poisson model is then a misspecification that usually *underestimates* standard errors.

```
var(bioChemists$art) / mean(bioChemists$art)
```

```
## [1] 2.191358
```

The variance is more than twice the mean. The **negative binomial** distribution counts the failures before the r-th success in independent trials with success probability θ:

$$P(x = k \mid r, \theta) = \binom{r + k - 1}{k} \theta^r (1 - \theta)^k = \frac{\Gamma(r + k)}{\Gamma(r)\, k!} \theta^r (1 - \theta)^k,$$

the Gamma form allowing non-integer r. In R, `dnbinom(2, 3, 0.75)` is $P(x = 2 \mid r = 3, \theta = 0.75)$. Its mean is $\mu = r(1 - \theta)/\theta$, so $\theta = r/(r + \mu)$ and the distribution can be parameterised by μ and the dispersion r; the variance $\mu + \mu^2/r$ exceeds the mean.

```
dnbinom(2, 3, 0.75)
```

```
## [1] 0.1582031
```

A negative binomial is a mixture of Poissons with gamma-distributed means, so it fits data arising from many Poisson processes with different rates.

Poisson–gamma mixture [Supplement]

If $\lambda \sim \text{Gamma}(\text{shape } r,\ \text{rate } r/\mu)$ and $x \mid \lambda \sim \text{Poisson}(\lambda)$, then

$$P(x = k) = \int_0^\infty \frac{e^{-\lambda}\lambda^k}{k!} \frac{(r/\mu)^r \lambda^{r-1} e^{-r\lambda/\mu}}{\Gamma(r)}\, d\lambda = \frac{\Gamma(r + k)}{\Gamma(r)\, k!} \left(\frac{r}{r + \mu}\right)^r \left(\frac{\mu}{r + \mu}\right)^k,$$

the negative binomial with $\theta = r/(r + \mu)$. The extra variance μ^2/r is the variance of the gamma-distributed rates.

Negative binomial regression assumes $y_i \sim \text{NegBin}(\mu_i, r)$ with $\log \mu_i = \beta_0 + \beta x_i$; `MASS::glm.nb()` fits it:

```
nb_mod <- glm.nb(art ~ gender, data = bioChemists)
tidy(nb_mod)
```

```
## # A tibble: 2 x 5
##   term        estimate std.error statistic  p.value
##   <chr>          <dbl>     <dbl>     <dbl>    <dbl>
```

```
## 1 (Intercept)    0.385    0.0544      7.08 1.44e-12
## 2 gender         0.247    0.0720      3.43 6.01e- 4
```

```
tidy(pois_mod)
```

```
## # A tibble: 2 x 5
##   term        estimate std.error statistic  p.value
##   <chr>          <dbl>     <dbl>     <dbl>    <dbl>
## 1 (Intercept)    0.385    0.0402      9.59 8.77e-22
## 2 gender         0.247    0.0519      4.77 1.89e- 6
```

The coefficients agree, but the negative binomial standard errors are larger: the Poisson model was overconfident.

6.3.3 Zero-inflated Poisson

Some counts have more zeros than a Poisson allows, e.g. the number of cigarettes smoked, where non-smokers always report 0. A **zero-inflated Poisson** (ZIP) model is a mixture: with probability θ_i the unit is a "structural zero" ($z_i = 1$), otherwise its count is Poisson,

$$y_i \sim \theta_i \mathbb{1}(y_i = 0) + (1 - \theta_i)\,\text{Poisson}(\lambda_i), \qquad \log \lambda_i = \beta_0 + \beta_1 x_i, \quad \text{logit}\,\theta_i = \gamma_0 + \gamma_1 x_i,$$

a Poisson regression for the counts and a logistic regression for the zeros. The slides fit cigarettes smoked against years of education (`zeroinfl(cigs ~ educ)`) and find that more educated people are more likely to be non-smokers ($\hat{\theta}$ rises from 0.49 at 6 years to 0.72 at 18) but, if they smoke, smoke more ($\hat{\lambda}$ from 18.3 to 27.7). With `bioChemists` (many students published nothing):

```
zip_mod <- zeroinfl(art ~ gender | gender, data = bioChemists)
zip_mod
```

```
##
## Call:
## zeroinfl(formula = art ~ gender | gender, data = bioChemists)
##
## Count model coefficients (poisson with log link):
## (Intercept)       gender
##      0.6151       0.2393
##
## Zero-inflation model coefficients (binomial with logit link):
## (Intercept)       gender
##    -1.35416     -0.03902
```

The count part says men publish more when they publish; the zero part estimates how gender affects the chance of being a "non-publisher".

6.4 Evaluating models

Motivation. With many possible models we need a way to choose. The fit to the data used for estimation is optimistic, because flexible models also fit the noise; honest evaluation needs data the model has not seen.

Predictive modelling looks for a function g with $y = g(x) + \epsilon$: in linear regression $g(x) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p$, in a GLM $g(x) = f^{-1}(\beta_0 + \cdots + \beta_p x_p)$. To compare specifications we look at bias and variance.

A coin example. To estimate p for a fair coin without data: model A picks a uniform random number (zero bias, large variance); model B always says 0.7 (zero variance, large bias). Neither is good; models generally trade bias against variance.

Proposition 6.1 (bias–variance decomposition)

For a new point (x, y) with $y = g(x) + \epsilon$, $\mathbb{E}\epsilon = 0$, and ϵ independent of the fitted model $\hat{y}(\cdot)$,

$$\mathbb{E}\big[(y - \hat{y}(x))^2\big] = \mathbb{E}[\epsilon^2] + \big(g(x) - \mathbb{E}[\hat{y}(x)]\big)^2 + \mathbb{E}\big[(\mathbb{E}[\hat{y}(x)] - \hat{y}(x))^2\big].$$

Proof [Supplement]. Write $y - \hat{y} = \epsilon + \{g(x) - \mathbb{E}\hat{y}\} + \{\mathbb{E}\hat{y} - \hat{y}\}$. Square and take expectations: the middle term is a constant; $\mathbb{E}\epsilon = 0$ and independence kill the cross terms involving ϵ; and $\mathbb{E}\{\mathbb{E}\hat{y} - \hat{y}\} = 0$ kills the cross term between the last two. ■

Model risk = observation variance + (model bias)^2 + model variance.

- *Observation variance* σ^2: irreducible noise (measurement error, missing information), beyond the analyst's control.
- *Model bias*: the gap between the average prediction and the truth, from a family too limited (a quadratic fitted to cubic data), i.e. **underfitting**; reduced by more complexity or domain knowledge.
- *Model variance*: the fitted model would change with a different sample; large when small differences in samples lead to large differences in fits, i.e. **overfitting**; reduced by less complexity.

As complexity increases, bias falls and variance rises, so the risk first decreases and then increases.

6.4.1 Training error and the holdout method

Measuring risk on the data used for fitting (training error) always favours more complex models. The simplest remedy is the **holdout method**: set aside part of the data. For `RailTrail`, hold out every fifth day (18 of 90) and fit polynomials of degree 1–5 on the other 72:

```
index <- (1:18) * 5
test <- RailTrail[index, ]
train <- RailTrail[-index, ]
mods <- map(1:5, ~ glm(volume ~ poly(hightemp, .x), data = train))
tibble(degree = 1:5,
       train_mse = map_dbl(mods, ~ mean((train$volume - predict(.x))^2)),
       valid_mse = map_dbl(mods, ~ mean((test$volume - predict(.x, newdata = test))^2)),
       AIC = map_dbl(mods, AIC))
```

```
## # A tibble: 5 x 4
##   degree train_mse valid_mse   AIC
##    <int>     <dbl>     <dbl> <dbl>
## 1      1    11429.     7835.  883.
## 2      2    10277.     6531.  877.
## 3      3     9318.     4975.  872.
## 4      4     9210.     5128.  874.
## 5      5     9203.     5023.  875.
```

Training error keeps falling with degree; validation error is smallest for degree 3, which AIC also selects (though by a different principle: a penalty instead of new data). Strictly, the held-out part is a *validation* set, not a test set. Common splits are 80/20, 90/10 or 75/25; there is no general rule.

6.4.2 Cross-validation

A single split wastes data and depends on the split. In k**-fold cross-validation** the data are divided at random into k folds; each fold in turn is the validation set for a model fitted on the other

$k-1$, giving $\text{MSE}_1, \ldots, \text{MSE}_k$, and

$$\text{CV}_{(k)} = \frac{1}{k}\sum_{i=1}^{k} \text{MSE}_i.$$

With $k = 2$: fit on one half, test on the other, then swap; a model that does well in-sample but poorly out-of-sample is overfit. **Leave-one-out** CV ($k = n$) has no randomness and less bias than the holdout method, but needs n fits. `boot::cv.glm()` computes it:

```
set.seed(1)
cv_error <- map_dbl(1:5, function(i) {
  fit <- glm(volume ~ poly(hightemp, i), data = RailTrail)
  boot::cv.glm(RailTrail, fit, K = 5)$delta[1]
})
cv_error
```

```
## [1] 11386.289  9845.353  8797.032  9319.328  9336.853
```

LOOCV without refitting [Supplement]

For least squares, leaving out observation i changes its residual to $e_i/(1-h_{ii})$, with h_{ii} the leverage, so $\text{CV}_{(n)} = \frac{1}{n}\sum_i \{e_i/(1-h_{ii})\}^2$ needs a single fit.

```
fit <- lm(volume ~ poly(hightemp, 3), data = RailTrail)
mean((residuals(fit) / (1 - hatvalues(fit)))^2)
```

```
## [1] 8992.758
```

```
boot::cv.glm(RailTrail, glm(volume ~ poly(hightemp, 3), data = RailTrail))$delta[1]
```

```
## [1] 8992.758
```

6.4.3 Why a separate test set

After comparing many models on the validation set, the best validation loss is an optimistically biased estimate of the chosen model's performance. Analogy: among millions of equally good golfers, the tournament winner was probably also a bit lucky; to estimate the winner's true distance, have them hit once more. Likewise, report performance on a **test set** that was never used for fitting or selection. The ideal workflow: training set to fit, validation set (or CV) to select, test set once at the end.

Lecture 7. Regularization

7.1 Beyond least squares

Motivation. With many predictors, least squares fits the noise as well as the signal and is hard to interpret. Fitting the same linear model in a smarter way, by choosing a subset of variables or by shrinking the coefficients, often predicts better and explains more simply.

The standard linear model $Y = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p + \epsilon$ extends to GLMs and to nonlinear models (splines, GAMs), yet linear models are often surprisingly competitive. Replacing least squares by other fitting procedures can improve prediction accuracy and interpretability. Three alternatives:

1. **subset selection**: use only some predictors;
2. **shrinkage** (regularization): shrink coefficients toward zero, reducing variance; some forms also select variables;
3. dimension reduction (not covered).

The same ideas apply to classification models.

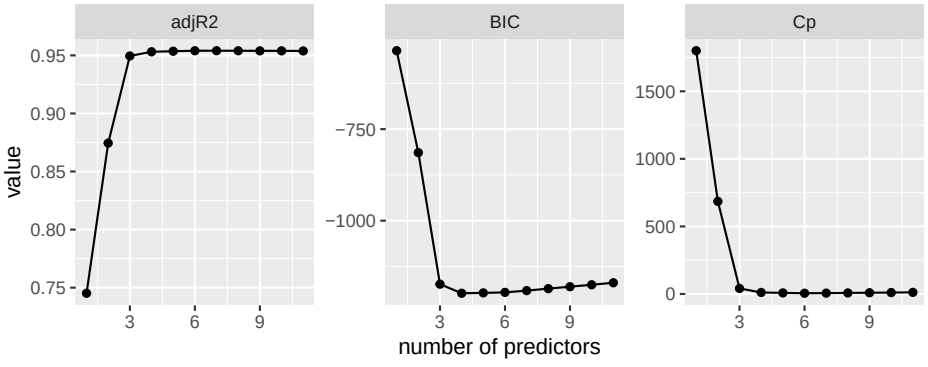
7.2 Subset selection

The fit of each candidate model is measured by its residual sum of squares $\text{RSS} = \sum_i (y_i - \hat{y}_i)^2$, the squared ($L_2$) loss.

Best subset selection fits all 2^p models: the null model M_0; for each $k = 1, \ldots, p$ all $\binom{p}{k}$ models with k predictors, keeping the best one M_k (smallest RSS); then chooses among $M_0, \ldots, M_p$. The last step cannot use RSS, which always decreases with size and would select the full model; it needs an estimate of *test* error: a validation set, C_p, BIC or adjusted R^2.

For the `ISLR2::Credit` data (credit card balance of 400 people), `leaps::regsubsets()` finds the best model of each size:

```
fit_full <- regsubsets(Balance ~ ., data = Credit, nvmax = 11)
s <- summary(fit_full)
tibble(d = 1:11, Cp = s$cp, BIC = s$bic, adjR2 = s$adjr2) %>%
  pivot_longer(-d) %>%
  ggplot(aes(d, value)) + geom_line() + geom_point() +
  facet_wrap(~ name, scales = "free_y") + labs(x = "number of predictors")
```




```
c(Cp = which.min(s$cp), BIC = which.min(s$bic), adjR2 = which.max(s$adjr2))
```

```
##    Cp   BIC adjR2
##     6     4     7
```

BIC chooses the smallest model; all three flatten out after about four predictors.

Stepwise selection. Best subset is infeasible for large p, and its huge search space also invites overfitting: the more models searched, the more likely one looks good on the training data by chance. **Forward stepwise** starts from the null model and adds, one at a time, the variable giving the largest improvement. It fits $1 + \sum_{k=0}^{p-1}(p-k) = 1 + p(p+1)/2$ models: for $p = 20$, 211 instead of $2^{20} = 1\,048\,576$, but it is not guaranteed to find the best model. **Backward stepwise** starts from the full model and removes the least useful variable at each step (requires $n > p$). **Hybrid** methods alternate: add a variable that improves the fit, then remove any that no longer help. The three approaches give similar but not identical models.

```
fwd <- regsubsets(Balance ~ ., data = Credit, nvmax = 11, method = "forward")
names(coef(fit_full, 4)); names(coef(fwd, 4))
```

```
## [1] "(Intercept)" "Income"      "Limit"       "Cards"       "StudentYes"
```

```
## [1] "(Intercept)" "Income"      "Limit"       "Rating"      "StudentYes"
```

Here the best 4-variable model (`Income`, `Limit`, `Cards`, `Student`) differs from the forward-stepwise one, which keeps `Rating` because it entered first.

7.2.1 Choosing the optimal model

Test error can be estimated *indirectly*, by adjusting the training error for the optimism due to overfitting, or *directly*, by validation or cross-validation. For a least-squares model with d predictors and $\hat\sigma^2$ the error variance estimated from the full model:

$$C_p = \frac{1}{n}(\mathrm{RSS} + 2d\hat\sigma^2), \qquad \mathrm{BIC} = \frac{1}{n}(\mathrm{RSS} + \log(n)\, d\hat\sigma^2), \qquad R^2_{\mathrm{adj}} = 1 - \frac{\mathrm{RSS}/(n-d-1)}{\mathrm{TSS}/(n-1)}.$$

- C_p adds the penalty $2d\hat\sigma^2$ because the training RSS underestimates test error. AIC, defined for any model fitted by maximum likelihood, is proportional to C_p for Gaussian linear models (up to constants).
- BIC replaces 2 by $\log n$; since $\log n > 2$ for $n > 7$, it penalises size more and selects smaller models.
- Adjusted R^2 is *maximised*; that is the same as minimising $\mathrm{RSS}/(n-d-1)$, which can rise when a useless variable is added.

Why $2d\hat\sigma^2$ [Supplement]

For least squares with d predictors (plus intercept), $\mathbb{E}[\mathrm{RSS}] = (n-d-1)\sigma^2$, while the expected squared error on new responses at the same x's is $(n+d+1)\sigma^2$. The difference is $2(d+1)\sigma^2$: the training error is optimistic by exactly twice the number of parameters times σ^2, which is what C_p adds back (the constant $2\sigma^2$ for the intercept does not affect comparisons).

Cross-validation estimates test error directly and needs no estimate of σ^2 or count of parameters, so it applies to any model.

7.3 Regularization

Motivation. Subset selection makes a yes–no decision for each variable, which is unstable and computationally hard. Regularization replaces it by a continuous penalty on the size of the coefficients,

which can be optimised efficiently and tuned smoothly.

Regularization adds constraints to a model $f_\beta(x)$ to prevent overfitting. We want to minimise the average loss $\frac{1}{n}\sum_i L(y_i, f_\beta(x_i))$ and to avoid unnecessarily complex models. If complexity is the number of nonzero coefficients,

$$\hat{\beta} = \arg\min_\beta \frac{1}{n}\sum_i L(y_i, f_\beta(x_i)) \quad \text{subject to} \quad \sum_{j=1}^{p} \mathbb{1}(\beta_j \neq 0) \leq c,$$

which is best subset selection: a combinatorial, in general NP-hard problem whose constraint (L_0 "norm") is far from convex. Convex substitutes:

- the L_1 ball $\sum_j |\beta_j| \leq c_1$ (the **lasso**), which encourages sparse solutions;
- the L_2 ball $\sum_j \beta_j^2 \leq c_2$ (**ridge**), which spreads weight over features but does not give zeros;
- the **elastic net**, a compromise between the two.

Do not confuse the L_2 *loss* with the L_2 *penalty*; the lasso combines L_2 loss with L_1 penalty.

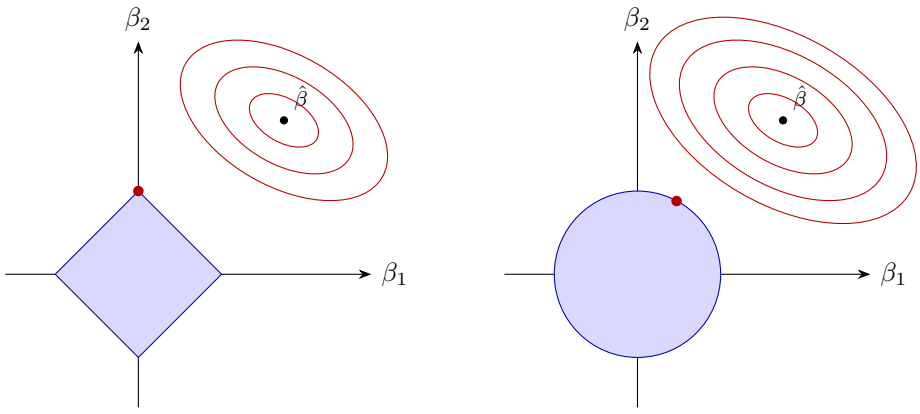


Figure. *Contours of the RSS (red ellipses) around the least-squares estimate $\hat{\beta}$ and the constraint regions. The constrained solution is where the smallest contour touches the region: at a corner of the diamond for the lasso (here $\beta_1 = 0$), at a generic point of the disc for ridge.* [Supplement]

7.3.1 Penalized form

With complexity $R(\beta)$, the constrained problem is equivalent to

$$\hat{\beta} = \arg\min_\beta \Big[\frac{1}{n}\sum_{i=1}^{n} L(y_i, \hat{y}_i) + \lambda R(\beta)\Big],$$

with a **regularization (tuning) parameter** $\lambda \geq 0$: ridge uses $\lambda\sum_j \beta_j^2$, the lasso $\lambda\sum_j |\beta_j|$ (the intercept is not penalised, and predictors are usually standardised first so the penalty treats them equally). The penalty is small when the coefficients are near zero, so it **shrinks** the estimates toward zero. With $\lambda = 0$ we get least squares; as $\lambda \to \infty$ the coefficients approach zero. Each λ gives a different estimate $\hat{\beta}_\lambda^R$, so choosing λ well is critical, and cross-validation is the standard tool. Shrinkage adds bias but lowers variance; for some λ the total test error is smaller than for least squares, which is why ridge and the lasso are particularly useful with many predictors.

Proposition 7.1 (orthonormal case)

Let $n = p$, $X = I$ and no intercept, so least squares minimises $\sum_j (y_j - \beta_j)^2$ with solution $\hat\beta_j = y_j$. Then ridge and lasso give

$$\hat\beta_j^{\text{ridge}} = \frac{y_j}{1+\lambda}, \qquad \hat\beta_j^{\text{lasso}} = \begin{cases} y_j - \lambda/2, & y_j > \lambda/2, \\ y_j + \lambda/2, & y_j < -\lambda/2, \\ 0, & |y_j| \le \lambda/2. \end{cases}$$

Proof [Supplement]. Both objectives are sums of one-dimensional problems. Ridge: $\frac{d}{d\beta}\{(y-\beta)^2 + \lambda\beta^2\} = -2(y-\beta) + 2\lambda\beta = 0$ gives $\beta = y/(1+\lambda)$. Lasso: $h(\beta) = (y-\beta)^2 + \lambda|\beta|$ is convex. For $\beta > 0$, $h'(\beta) = -2(y-\beta) + \lambda$ vanishes at $\beta = y - \lambda/2$, admissible iff $y > \lambda/2$; symmetrically for $\beta < 0$. If $|y| \le \lambda/2$, the one-sided derivatives at 0 are $-2y + \lambda \ge 0$ (right) and $-2y - \lambda \le 0$ (left), so 0 is the minimum. ■

Ridge shrinks every coefficient by the same proportion; the lasso shifts each toward zero by $\lambda/2$ and sets small ones exactly to zero (**soft thresholding**), which is why it performs variable selection.

7.4 Example: delays of flights to California

Arrival delays of flights from New York (EWR, JFK) to four California airports, explained by hour, origin, destination, carrier, month and day of week. The slides use `tidymodels`; the same models are fitted here with `lm()` and `glmnet()` directly.

```
California <- flights %>%
  filter(dest %in% c("LAX", "SFO", "OAK", "SJC"), !is.na(arr_delay)) %>%
  mutate(day = as.Date(time_hour),
         dow = as.character(lubridate::wday(day, label = TRUE)),
         month = as.factor(month), hour = as.factor(hour))
dim(California)
```

```
## [1] 29836    20
```

```
set.seed(386)
idx <- sample(nrow(California), size = round(0.7 * nrow(California)))
train <- California[idx, ]; test <- California[-idx, ]
flight_model <- arr_delay ~ origin + dest + hour + carrier + month + dow
mod_reg <- lm(flight_model, data = train)
tidy(mod_reg) %>% head(4)
```

```
## # A tibble: 4 x 5
##   term        estimate std.error statistic  p.value
##   <chr>          <dbl>     <dbl>     <dbl>    <dbl>
## 1 (Intercept)   -16.0      6.12      -2.61 0.00905
## 2 originJFK       2.88     0.783      3.67 0.000239
## 3 destOAK        -4.61     3.10      -1.49 0.136
## 4 destSFO         1.89     0.620      3.05 0.00227
```

```
rmse <- function(y, yhat) sqrt(mean((y - yhat)^2))
rmse(test$arr_delay, predict(mod_reg, newdata = test))
```

```
## [1] 42.60868
```

`glmnet()` needs a numeric design matrix; `alpha = 1` is the lasso, `alpha = 0` ridge, and values in between the elastic net (the `mixture` argument in tidymodels):

```
X_train <- model.matrix(flight_model, data = train)[, -1]
X_test <- model.matrix(flight_model, data = test)[, -1]
mod_lasso <- glmnet(X_train, train$arr_delay, alpha = 1)
```

```
coef(mod_lasso, s = 0.01)[1:4, ]
```

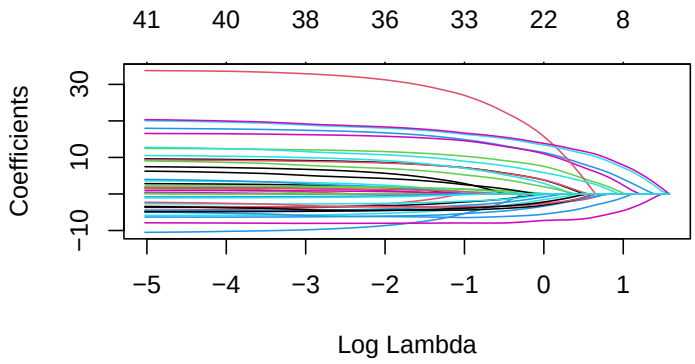
```
## (Intercept)   originJFK     destOAK     destSFO
##  -11.406671    2.783746   -4.399701    1.879023
```

```
rmse(test$arr_delay, predict(mod_lasso, newx = X_test, s = 0.01))
```

```
## [1] 42.60797
```

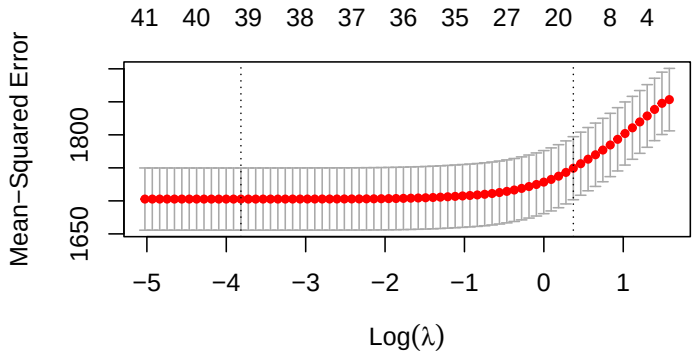
With a small penalty the lasso barely changes the coefficients (`originJFK` moves slightly toward zero) and does not improve test error here: with about 30 000 flights and 40 predictors the least-squares fit is not overfit. With many more predictors relative to n, regularization typically helps. The coefficient paths show each β_j shrinking to zero as λ grows:

```
plot(mod_lasso, xvar = "lambda")
```



Choosing λ by 10-fold cross-validation (`cv.glmnet()`), with the "one standard error" rule giving a sparser model:

```
set.seed(1)
cv_lasso <- cv.glmnet(X_train, train$arr_delay, alpha = 1)
plot(cv_lasso)
```




```
c(lambda.min = cv_lasso$lambda.min, lambda.1se = cv_lasso$lambda.1se)
```

```
## lambda.min lambda.1se
##  0.0220299  1.4494202
```

```
sum(coef(cv_lasso, s = "lambda.1se") != 0)
```

```
## [1] 19
```

```
rmse(test$arr_delay, predict(cv_lasso, newx = X_test, s = "lambda.1se"))
```

```
## [1] 43.34445
```

Ridge ($\alpha = 0$) keeps all predictors but shrinks them:

```
mod_ridge <- glmnet(X_train, train$arr_delay, alpha = 0)
coef(mod_ridge, s = 0.1)[1:4, ]
```

```
## (Intercept)   originJFK     destOAK     destSFO
##   -7.683246    2.725897   -4.448581    1.861059
```

Lecture 8. Classification I

8.1 Supervised learning

Motivation. Many business and scientific decisions are classifications: is this e-mail phishing, will this customer leave, will this user click? Classification is prediction with a categorical outcome, and the same logic of training on labelled data and checking on new data applies.

Data scientists are often asked to automate decisions: Is an e-mail a phishing attempt? Is a customer likely to churn? Will a user click on an ad? These are **classification** problems, a form of **supervised learning**: train a model on data where the outcome is known, then apply it where it is not. Machine learning algorithms predict, classify or cluster; prediction (numeric response) and classification (categorical response) are supervised, clustering (no response) is **unsupervised**.

Prediction. In `RailTrail`, a LOESS smoother of `volume` on `avgtemp` is a prediction model: the response "supervises" the fit, and the fitted values are predictions.

```
RailTrail %>%
  mutate(loess.fit = loess(volume ~ avgtemp, data = .)$fitted) %>%
  select(volume, loess.fit, avgtemp) %>% head(4)
```

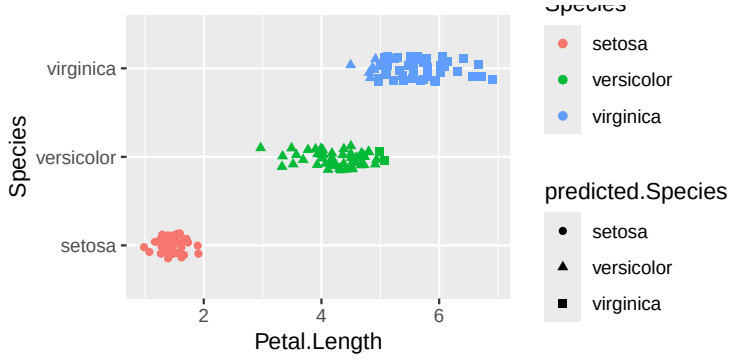
```
##   volume loess.fit avgtemp
## 1    501  469.4541    66.5
## 2    419  443.2781    61.0
## 3    397  462.1633    63.0
## 4    385  400.6197    78.0
```

Classification. In `iris`, petal length alone almost separates the three species. A crude rule read off a plot, setosa below 2, versicolor between 2 and 5, virginica above 5, already classifies well:

```
iris.crude <- iris %>%
  mutate(predicted.Species = ifelse(Petal.Length < 2, "setosa",
                                    ifelse(Petal.Length < 5, "versicolor", "virginica")))
iris.crude %>% count(Species, predicted.Species)
```

```
##      Species predicted.Species  n
## 1     setosa            setosa 50
## 2 versicolor        versicolor 48
## 3 versicolor         virginica  2
## 4  virginica        versicolor  6
## 5  virginica         virginica 44
```

```
ggplot(iris.crude, aes(Petal.Length, Species, colour = Species, shape = predicted.
    Species)) +
  geom_jitter(height = 0.15)
```



Classification is a special case of prediction: `predicted.Species` predicts `Species` and takes the same set of values. With two variables (petal length and width) we can still look; to use all

four, or many more, we need model- or algorithm-based methods. Visualisation remains useful for exploration and for communicating results.

8.2 Naive Bayes

Motivation. Bayes' theorem turns probabilities of the predictors given the class, which are easy to estimate, into the probability of the class given the predictors, which is what we want. With many predictors one strong simplification makes this practical.

Example. The slides use the 1994 US Census (32 561 adults; 24% earn more than \$50 000). With $y = 1$ for high earners and $x = 1$ for male, $P(y = 1) = 0.241$, $P(\text{male}) = 0.670$ and $P(\text{male} \mid y = 1) = 0.845$, so by Bayes' theorem

$$P(y = 1 \mid \text{male}) = \frac{P(\text{male} \mid y = 1)\, P(y = 1)}{P(\text{male})} = \frac{0.845 \times 0.241}{0.670} \approx 0.30.$$

Marginal probabilities are usually easy to compute, so knowing $P(y \mid x)$ is equivalent to knowing $P(x \mid y)$. A classifier assigns x^* to $y = 1$ when $P(y = 1 \mid x^*) > P(y = 0 \mid x^*)$.

With several predictors, regression models $P(y \mid x_1, \ldots, x_p)$ directly; naive Bayes instead models $P(x_1, \ldots, x_p \mid y)$ under the (overly) simple assumption that the predictors are **conditionally independent** given the class:

$$P(x_1, \ldots, x_p \mid y) = \prod_{j=1}^{p} P(x_j \mid y),$$

each factor being easy to estimate since y takes few values. By Bayes' theorem, for classes $k = 1, \ldots, K$,

$$P(y = i \mid x_1, \ldots, x_p) = \frac{\prod_j P(x_j \mid y = i)\; P(y = i)}{\sum_{k=1}^{K} \prod_j P(x_j \mid y = k)\; P(y = k)}.$$

The labels $1, \ldots, K$ carry no order; numeric predictors need a model for $P(x_j \mid y)$, usually a normal density. There is no regression, only the conditional independence assumption, yet naive Bayes often works well.

The census data must be downloaded from the UCI repository, so here we use `ISLR2::Default` (10 000 credit-card customers; did they default?) for all classifiers in this lecture. Split 80/20 into training and test sets:

```
set.seed(364)
idx <- sample(nrow(Default), 0.8 * nrow(Default))
train <- Default[idx, ]; test <- Default[-idx, ]
mean(train$default == "Yes")
```

```
## [1] 0.03125
```

```
accuracy <- function(pred, truth) mean(pred == truth)
mod_nb <- naiveBayes(default ~ student + balance + income, data = train)
accuracy(predict(mod_nb, test), test$default)
```

```
## [1] 0.9625
```

Only about 3% default, so always predicting "No" would already be 97% accurate: accuracy alone is a poor measure with unbalanced classes (Lecture 9 introduces better ones) [Supplement].

8.3 Discriminant analysis

Motivation. If each class's predictors are roughly normal, Bayes' theorem gives the optimal rule in closed form. Linear discriminant analysis is that rule with a common covariance, and its decision

boundaries are straight lines.

Discriminant analysis, introduced by R. A. Fisher in 1936, is the oldest statistical classifier. **Linear discriminant analysis** (LDA) is less common now that tree methods exist, but still appears in applications.

One predictor. Assume $p(x \mid y = k) = f_k(x) = \phi(x; \mu_k, \sigma^2)$, normal with class-specific means and a common variance, with prior $\pi_k = P(y = k)$. Plugging in the estimates $\hat\pi_k = n_k/n$, $\hat\mu_k$, $\hat\sigma^2$, the Bayes rule assigns x to the class with the largest **discriminant function**

$$\delta_k(x) = x\,\frac{\hat\mu_k}{\hat\sigma^2} - \frac{\hat\mu_k^2}{2\hat\sigma^2} + \log\hat\pi_k.$$

Proposition 8.1 (LDA discriminant)

Under the model above, $P(y = k \mid x)$ is largest for the k maximising $\delta_k(x)$. With two classes and $\pi_0 = \pi_1$, the boundary is $x = (\mu_0 + \mu_1)/2$.

Proof [Supplement]. $P(y = k \mid x) \propto \pi_k f_k(x)$, with the same denominator for all k. Taking logs, $\log\pi_k - \frac{(x-\mu_k)^2}{2\sigma^2} + \text{const} = \log\pi_k + \frac{x\mu_k}{\sigma^2} - \frac{\mu_k^2}{2\sigma^2} - \frac{x^2}{2\sigma^2} + \text{const}$; the last two terms are common to all classes. With equal priors, $\delta_1(x) > \delta_0(x)$ iff $x(\mu_1 - \mu_0)/\sigma^2 > (\mu_1^2 - \mu_0^2)/(2\sigma^2)$, i.e. (for $\mu_1 > \mu_0$) iff $x > (\mu_0 + \mu_1)/2$. ■

Several predictors. Each class is multivariate normal $N(\mu_k, \Sigma)$ with a common covariance matrix and density

$$f(x) = \frac{1}{(2\pi)^{p/2}|\Sigma|^{1/2}} \exp\Big\{-\frac{1}{2}(x-\mu_k)^{\mathsf T}\Sigma^{-1}(x-\mu_k)\Big\}.$$

(Contours are circles when the variables are uncorrelated with equal variances, tilted ellipses when correlated.) Taking logs and dropping terms common to all classes gives

$$\delta_k(x) = x^{\mathsf T}\hat\Sigma^{-1}\hat\mu_k - \frac{1}{2}\hat\mu_k^{\mathsf T}\hat\Sigma^{-1}\hat\mu_k + \log\hat\pi_k,$$

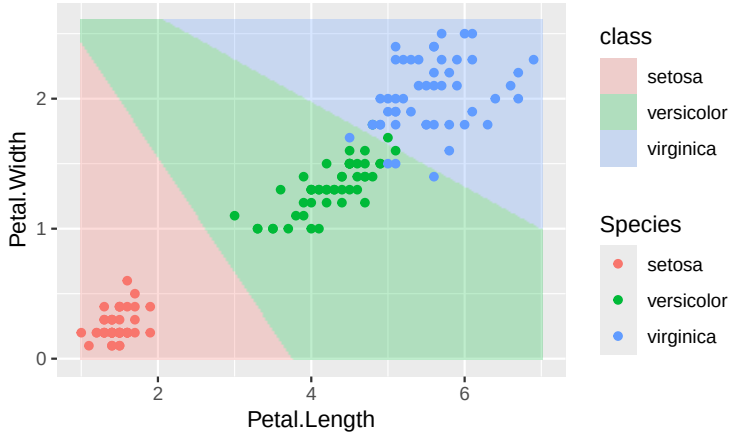
linear in x; LDA assigns x to the class with the largest $\delta_k(x)$, for any number of classes.

QDA. Allowing a different covariance Σ_k for each class gives **quadratic discriminant analysis**, with quadratic boundaries: more flexible, with more parameters. LDA often performs well in practice and similarly to logistic regression.

```
mod_lda <- lda(default ~ student + balance + income, data = train)
mod_qda <- qda(default ~ student + balance + income, data = train)
c(lda = accuracy(predict(mod_lda, test)$class, test$default),
  qda = accuracy(predict(mod_qda, test)$class, test$default))
```

```
##    lda    qda
## 0.9655 0.9680
```

```
lda2 <- lda(Species ~ Petal.Length + Petal.Width, data = iris)
grid <- expand_grid(Petal.Length = seq(1, 7, length = 200), Petal.Width = seq(0, 2.6,
    length = 200))
grid$class <- predict(lda2, grid)$class
ggplot(iris, aes(Petal.Length, Petal.Width)) +
  geom_raster(data = grid, aes(fill = class), alpha = 0.25) +
  geom_point(aes(colour = Species))
```



LDA on two iris measurements: the decision boundaries between the three classes are straight lines.

8.4 Logistic regression for classification

Motivation. Logistic regression (Lecture 6) gives probabilities; a threshold turns them into classes. The more interesting question is how its coefficients are fitted: least squares fails here, and the right loss, cross-entropy, turns out to be maximum likelihood.

The model $\operatorname{logit}(q) = \log \frac{q}{1-q} = x^{\mathsf{T}}\beta$ gives $q = P(y = 1 \mid x) = \operatorname{expit}(x^{\mathsf{T}}\beta) = 1/(1 + e^{-x^{\mathsf{T}}\beta})$. Classifying $y = 1$ when $q > 0.5$ means $x^{\mathsf{T}}\beta > 0$: the **decision boundary** is the hyperplane $x^{\mathsf{T}}\beta = 0$.

High earners. With capital gains (taxes paid on gains from assets, which indicate wealth and are part of income) as the only predictor, the slides obtain $\hat{\beta}_0 = -1.37$ and $\hat{\beta}_1 = 0.000335$; the boundary $-\hat{\beta}_0/\hat{\beta}_1 \approx \$4\,102$ means: classify a taxpayer as high-income if their capital gains exceed $4 102. The same on `Default` with balance:

```
mod_log_1 <- glm(default ~ balance, family = binomial, data = train)
tidy(mod_log_1)
```

```
## # A tibble: 2 x 5
##   term         estimate std.error statistic   p.value
##   <chr>           <dbl>     <dbl>     <dbl>     <dbl>
## 1 (Intercept) -10.8      0.421        -25.8 1.56e-146
## 2 balance       0.00559  0.000257      21.8 3.12e-105
```

```
-coef(mod_log_1)[1] / coef(mod_log_1)[2]
```

```
## (Intercept)
##     1939.69
```

A customer is classified as a defaulter when the balance exceeds about $1 940.

8.4.1 Why not squared loss

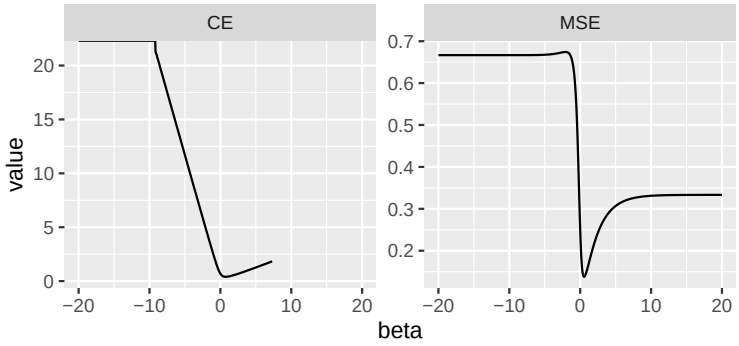
In linear regression the coefficients minimise the mean squared error. Applied to logistic regression,

$$\frac{1}{n}\sum_{i=1}^{n}\left(y_i - \operatorname{expit}(x_i^{\mathsf{T}}\beta)\right)^2$$

compares a binary y with a probability, and is not convex in β. For a toy data set with a single coefficient ($\operatorname{logit} q = \beta x$):

```
x <- c(-4, -2, -0.5, 1, 3, 5); y <- c(0, 0, 1, 0, 1, 1)
expit <- function(z) 1 / (1 + exp(-z))
tibble(beta = seq(-20, 20, by = 0.05)) %>%
  mutate(MSE = map_dbl(beta, ~ mean((y - expit(.x * x))^2)),
         CE = map_dbl(beta, ~ -mean(y * log(expit(.x * x)) + (1 - y) * log(1 - expit(.x
    * x))))) %>%
  pivot_longer(-beta) %>%
  ggplot(aes(beta, value)) + geom_line() + facet_wrap(~ name, scales = "free_y")
```

```
## Warning: Removed 254 rows containing missing values or values outside the scale range
## (`geom_line()`).
```




```
mse <- function(b) mean((y - expit(b * x))^2)
sapply(c(-20, -10.34, -2, 0.54, 5), mse)
```

```
## [1] 0.6666667 0.6666720 0.6741982 0.1378328 0.3067829
```

```
optimize(mse, c(-5, 5))$minimum
```

```
## [1] 0.5446891
```

The MSE curve (right) is not convex: it has a hump near $\beta = -2$ and for negative β becomes an almost flat plateau at 2/3. A gradient method started on the left keeps moving away and stops where the slope is numerically zero (the slides report such a point, $\beta = -10.34$), far from the global minimum $\beta \approx 0.54$. Adding variables does not fix this, and the squared error of a probability is bounded by 1, which is conceptually questionable.

8.4.2 Cross-entropy loss

For binary y and predicted probability q, the **cross-entropy** (log) loss is

$$-\{y \log q + (1-y)\log(1-q)\}.$$

If $y = 1$ the loss is $-\log q$, zero when $q \to 1$ and infinite when $q \to 0$; if $y = 0$ it is $-\log(1-q)$. It strongly penalises confident wrong predictions and, averaged over the data,

$$-\frac{1}{n}\sum_{i=1}^{n}\left\{y_i \log \operatorname{expit}(x_i^\top \beta) + (1-y_i)\log\left(1 - \operatorname{expit}(x_i^\top \beta)\right)\right\},$$

is convex in β (left panel above), so it has a single minimum.

Convexity of cross-entropy [Supplement]

With $q_i = \operatorname{expit}(x_i^\top \beta)$, the gradient of the average loss is $\frac{1}{n}\sum_i (q_i - y_i)x_i$ and the Hessian is $\frac{1}{n}\sum_i q_i(1-q_i)x_i x_i^\top$, a positive semidefinite matrix. Hence the loss is convex, and New-

ton's method (iteratively reweighted least squares, what `glm()` does) converges to the global minimum when it exists.

Connection to maximum likelihood. Given x_i, $y_i \sim \text{Bernoulli}(q_i)$ with likelihood $q_i^{y_i}(1-q_i)^{1-y_i}$, so the log-likelihood is

$$\sum_{i=1}^{n} \{y_i \log q_i + (1 - y_i) \log(1 - q_i)\},$$

exactly $-n$ times the average cross-entropy. Minimising cross-entropy *is* maximum likelihood estimation. Many "model + loss" combinations can be motivated this way: least squares (normal errors), and ridge and lasso (adding a prior on β).

More than two classes. Logistic regression extends to $K > 2$ classes, e.g. a chain of binary models ("1" vs "2 or 3", then "2" vs "3"), or the multinomial model (`nnet::multinom()`), but discriminant analysis is more often used for multiple classes.

Summary. Naive Bayes assumes conditional independence of the predictors; discriminant analysis (LDA, QDA) assumes multivariate normal predictors within classes; logistic regression assumes a linear model for logit $P(y = 1 \mid x)$, fitted by cross-entropy, i.e. maximum likelihood.

Lecture 9. Classification II

9.1 Logistic regression as a classifier

Motivation. A logistic model gives probabilities; turning them into decisions requires a threshold, and judging the decisions requires more than the share of correct answers. This lecture is about evaluating classifiers honestly, especially when one class is rare.

Review: logistic regression predicts $P(y = 1 \mid x) = \operatorname{expit}(x^{\mathsf{T}}\beta)$, is fitted with cross-entropy loss, and classifies $y = 1$ when $\hat{p} \geq 0.5$.

A coin example. Ten i.i.d. tosses give $0, 0, 0, 1, 1, 0, 0, 0, 1, 1$. The best constant model $P(y = 1) = \theta$ is the sample mean 0.4, because it maximises the likelihood $\theta^4(1-\theta)^6$, i.e. $4\log\theta + 6\log(1-\theta)$, whose derivative $4/\theta - 6/(1-\theta)$ vanishes at $\theta = 0.4$; equivalently it minimises the cross-entropy loss.

The null model. The slides use the census data (24% high earners). We continue with `ISLR2::Default` (credit-card default, about 3% "Yes"), split 80/20 as in Lecture 8. The null model $P(y = 1) = \operatorname{expit}(\beta_0)$ predicts the same probability $\bar{\pi}$ for everyone; since $\bar{\pi} < 0.5$, everyone is classified as $y = 0$.

```
set.seed(364)
idx <- sample(nrow(Default), 0.8 * nrow(Default))
train <- Default[idx, ]; test <- Default[-idx, ]
train %>% count(default) %>% mutate(pct = n / sum(n))
```

```
##   default    n     pct
## 1      No 7750 0.96875
## 2     Yes  250 0.03125
```

9.1.1 Thresholding

The decision rule need not use 0.5:

$$\text{classify}(x) = \begin{cases} 1, & \hat{P}(y = 1 \mid x) \geq t, \\ 0, & \text{otherwise.} \end{cases}$$

A logistic regression paired with a threshold is a classifier. Raising t classifies fewer observations as 1: fewer false positives but more false negatives.

9.1.2 Accuracy

The simplest evaluation is **accuracy**, the proportion of observations classified correctly. Comparing the null model, a model with `balance` only, and a model with all predictors:

```
fit_null <- glm(default ~ 1, family = binomial, data = train)
fit_1 <- glm(default ~ balance, family = binomial, data = train)
fit_all <- glm(default ~ student + balance + income, family = binomial, data = train)
classify <- function(fit, data, t = 0.5)
  factor(ifelse(predict(fit, data, type = "response") >= t, "Yes", "No"), levels = c("No
    ", "Yes"))
map_dbl(list(null = fit_null, bal = fit_1, all = fit_all),
        ~ mean(classify(.x, train) == train$default))
```

```
##     null      bal      all
## 0.968750 0.974125 0.974875
```

Adding predictors improves training accuracy, but only slightly, because the null model is already 97% accurate.

9.2 Evaluating classifiers

9.2.1 Drawbacks of accuracy

A spam filter applied to 100 e-mails, 2 spam and 98 ham, that labels everything as ham is 98% accurate and useless. With imbalanced classes accuracy is misleading.

9.2.2 The confusion matrix

Call class 1 positive and class 0 negative.

	predicted 1	predicted 0
actual 1	TP (true positive)	FN (false negative: missed)
actual 0	FP (false positive: false alarm)	TN (true negative)

$$\text{accuracy} = \frac{TP + TN}{n}, \qquad \text{precision} = \frac{TP}{TP + FP}, \qquad \text{recall} = \frac{TP}{TP + FN}.$$

Precision: of the observations predicted 1, the proportion that are actually 1. **Recall**: of the actual 1s, the proportion predicted 1. For the all-ham spam filter, TP = FP = 0, so precision is undefined (0/0) and recall is $0/2 = 0$.

```
table(predicted = classify(fit_1, test), actual = test$default)
```

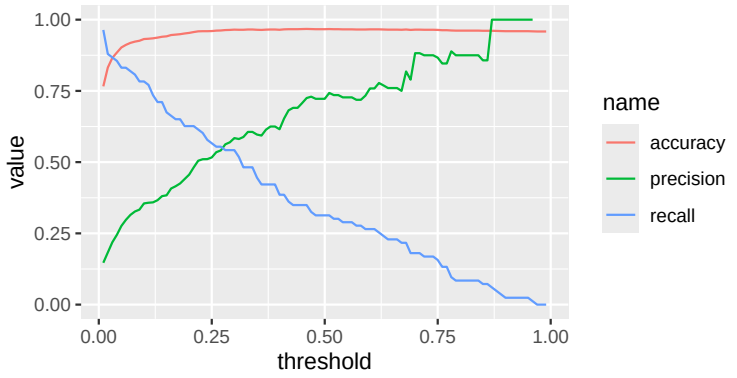
```
##          actual
## predicted   No  Yes
##       No  1907   58
##       Yes   10   25
```

The trade-off. Precision penalises false positives and recall penalises false negatives. A threshold of 0 (classify everyone as 1) gives 100% recall but low precision. Higher thresholds give fewer FP and higher precision; lower thresholds give fewer FN and higher recall. Accuracy is typically highest in between.

Choosing a metric depends on the domain. For screening a disease, maximise recall: a false negative means no treatment, while a false positive only leads to further tests. For deciding a life sentence, maximise precision: a false positive puts an innocent person in prison.

Other names: **sensitivity** = recall = true positive rate (TPR); **specificity** = true negative rate $= TN/(TN + FP)$; 1−specificity = false positive rate (FPR); precision = positive predictive value (PPV).

```
p_test <- predict(fit_all, test, type = "response")
y_test <- test$default == "Yes"
tibble(t = seq(0.01, 0.99, by = 0.01)) %>%
  mutate(accuracy = map_dbl(t, ~ mean((p_test >= .x) == y_test)),
         precision = map_dbl(t, ~ sum(p_test >= .x & y_test) / sum(p_test >= .x)),
         recall = map_dbl(t, ~ sum(p_test >= .x & y_test) / sum(y_test))) %>%
  pivot_longer(-t) %>%
  ggplot(aes(t, value, colour = name)) + geom_line(na.rm = TRUE) + labs(x = "threshold")
```



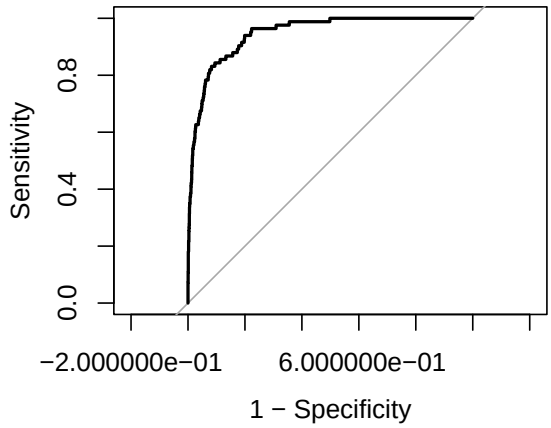
9.2.3 ROC and precision–recall curves

Plotting precision against recall over all thresholds gives the **precision–recall (PR) curve**; the perfect classifier reaches precision 1 and recall 1. Plotting sensitivity (TPR) against 1−specificity (FPR) gives the **ROC** (receiver operating characteristic) curve, standard in machine learning, operations research and medical testing. We want the ROC curve as close to the top-left corner as possible; the **area under the curve** (AUC) summarises it: 1 for a perfect classifier, 0.5 for random guessing. The PR curve is more informative when the classes are highly unbalanced.

```
roc_1 <- roc(test$default, predict(fit_1, test, type = "response"), quiet = TRUE)
roc_all <- roc(test$default, p_test, quiet = TRUE)
c(auc_balance = auc(roc_1), auc_all = auc(roc_all))
```

```
## auc_balance     auc_all
##   0.9468359   0.9467919
```

```
plot(roc_all, legacy.axes = TRUE)
```



What AUC measures [Supplement]

AUC equals the probability that a randomly chosen positive case receives a higher predicted probability than a randomly chosen negative case (the Mann–Whitney statistic). It therefore measures ranking quality and does not depend on the threshold or on the class proportions.

9.2.4 Training versus testing

With the census data the slides compare three models:

model	accuracy (train)	accuracy (test)	sensitivity (test)	specificity (test)
all predictors	0.852	0.849	0.598	0.928
capital gain only	0.800	0.803	0.204	0.991
null	0.759	0.761	0.000	1.000

Each model does slightly worse on the test set. The null model has sensitivity 0 and specificity 1, since it always predicts the same class; the full model is a little less specific but much more sensitive, so it is the most useful. For `Default`:

```
eval_model <- function(fit) {
  pr <- classify(fit, test)
  c(acc_train = mean(classify(fit, train) == train$default),
    acc_test = mean(pr == test$default),
    sens = mean(pr[test$default == "Yes"] == "Yes"),
    spec = mean(pr[test$default == "No"] == "No"))
}
round(rbind(all = eval_model(fit_all), balance = eval_model(fit_1), null = eval_model(
    fit_null)), 3)
```

```
##         acc_train acc_test  sens  spec
## all         0.975    0.967 0.313 0.995
## balance     0.974    0.966 0.301 0.995
## null        0.969    0.959 0.000 1.000
```

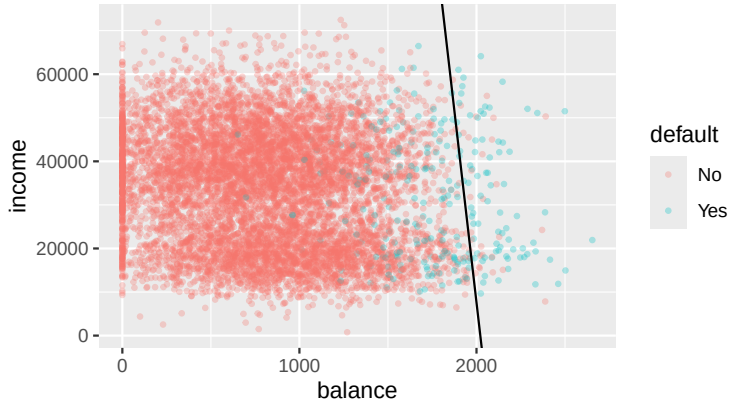
At threshold 0.5 the sensitivity is low: most defaulters are missed. If missing a defaulter is costly, a lower threshold is appropriate.

9.3 Linear separability and regularization

Motivation. With one predictor the decision boundary is a point, with two a line. When a line separates the classes perfectly, maximum likelihood breaks down: the coefficients run off to infinity. Regularization fixes this.

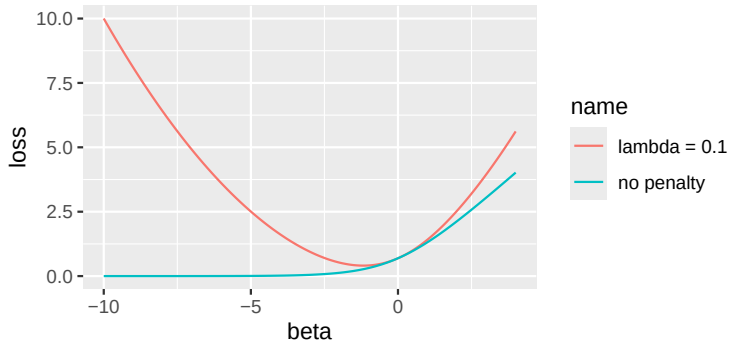
Decision boundaries. With hours worked per week as the only predictor, the slides obtain $\operatorname{logit}\hat{\pi} = -3.085 + 0.046 \cdot \text{hours}$; at threshold 0.5 the boundary is $-3.085 + 0.046 \cdot \text{hours} = 0$, i.e. about 67 hours. With two predictors, $\hat{\pi} = \operatorname{expit}(-4.85 + 0.047 \cdot \text{hours} + 0.042 \cdot \text{age})$, the boundary is a line in the (hours, age) plane; points of class 1 on the class-0 side are false negatives and vice versa.

```
fit_2 <- glm(default ~ balance + income, family = binomial, data = train)
b <- coef(fit_2)
ggplot(train, aes(balance, income, colour = default)) +
  geom_point(alpha = 0.3, size = 0.8) +
  geom_abline(intercept = -b[1] / b[3], slope = -b[2] / b[3])
```



Perfect separation. Take two points, $(x, y) = (-1, 1)$ and $(1, 0)$, and the model $\text{expit}(\beta_1 x)$. The mean cross-entropy loss decreases monotonically to 0 as $\beta_1 \to -\infty$, so the "best" estimate is $\hat{\beta}_1 = -\infty$, which predicts probabilities of exactly 0 or 1. That is overconfident: a new observation $(0.5, 1)$ would get predicted probability 0 and *infinite* loss.

```
x <- c(-1, 1); y <- c(1, 0); expit <- function(z) 1 / (1 + exp(-z))
ce <- function(b, lambda = 0)
  -mean(y * log(expit(b * x)) + (1 - y) * log(1 - expit(b * x))) + lambda * b^2
tibble(beta = seq(-10, 4, by = 0.05)) %>%
  mutate(`no penalty` = map_dbl(beta, ce), `lambda = 0.1` = map_dbl(beta, ce, lambda =
    0.1)) %>%
  pivot_longer(-beta) %>%
  ggplot(aes(beta, value, colour = name)) + geom_line() + labs(y = "loss")
```



Points are **linearly separable** if a line (in d dimensions, a $(d-1)$-dimensional hyperplane) separates the classes perfectly. Then some coefficients diverge; in practice the software stops at excessively large values (and `glm()` warns that fitted probabilities 0 or 1 occurred).

Regularized logistic regression adds a penalty, e.g. L_2:

$$-\frac{1}{n}\sum_{i=1}^{n}\left\{y_i \log \text{expit}(x_i^\top \beta) + (1-y_i)\log\left(1-\text{expit}(x_i^\top \beta)\right)\right\} + \lambda \sum_{j=1}^{p} \beta_j^2.$$

The penalised loss (curve above) has a finite minimum even for separable data. As with ridge and lasso, standardise the predictors first. In R, `glmnet(..., family = "binomial")` fits it:

```
X <- scale(model.matrix(default ~ student + balance + income, data = train)[, -1])
cv_fit <- cv.glmnet(X, train$default, family = "binomial", alpha = 0)
coef(cv_fit, s = "lambda.min")
```

```
## 4 x 1 sparse Matrix of class "dgCMatrix"
##                      s1
## (Intercept) -4.83965221
## studentYes  -0.11306543
## balance      1.81973290
## income       0.01566196
```

Lecture 10. Splines and Generalized Additive Models

10.1 Beyond linearity

Motivation. Linear models are simple to fit and interpret, and regularization improves them further, but some relationships are clearly curved. The methods here relax linearity one step at a time while keeping as much of the linear model's interpretability as possible.

Linear models are simple to describe, implement and interpret, and the linearity assumption is often a good approximation. Regularization (Lecture 7) reduces their variance. To relax linearity itself while staying interpretable we consider polynomial regression and step functions, regression and smoothing splines, local regression, and generalized additive models (GAMs). The running example is `ISLR2::Wage`: wages of 3000 male workers in the US Mid-Atlantic region.

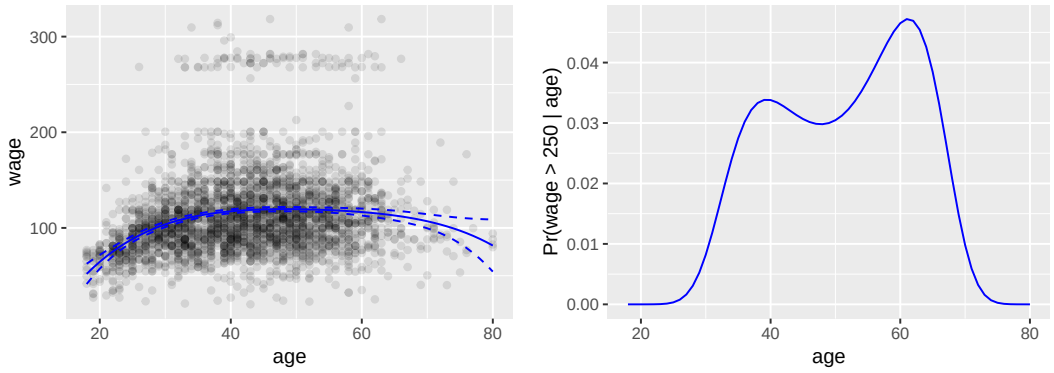
10.2 Polynomial regression and step functions

Polynomial regression replaces the line by

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \cdots + \beta_d x_i^d + \epsilon_i,$$

still linear in the coefficients, hence fitted by least squares. Degrees above 4 are rarely used: the curve becomes overly flexible, with strange shapes near the boundary of X. The individual coefficients are not of interest; we look at the fitted function $\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0 + \cdots + \hat{\beta}_4 x_0^4$ and its pointwise confidence band (± 2 standard errors).

```
fit4 <- lm(wage ~ poly(age, 4), data = Wage)
grid <- tibble(age = 18:80)
pr <- predict(fit4, grid, se.fit = TRUE)
grid <- grid %>% mutate(fit = pr$fit, lo = fit - 2 * pr$se.fit, hi = fit + 2 * pr$se.fit
    )
ggplot(Wage, aes(age, wage)) + geom_point(alpha = 0.1) +
  geom_line(data = grid, aes(y = fit), colour = "blue") +
  geom_line(data = grid, aes(y = lo), colour = "blue", linetype = "dashed") +
  geom_line(data = grid, aes(y = hi), colour = "blue", linetype = "dashed")
fit_log <- glm(I(wage > 250) ~ poly(age, 4), family = binomial, data = Wage)
grid %>% mutate(p = predict(fit_log, grid, type = "response")) %>%
  ggplot(aes(age, p)) + geom_line(colour = "blue") + labs(y = "Pr(wage > 250 | age)")
```



Wages seem to come from two populations, a high-earner group above \$250 000 and the rest. Treating $\mathbb{1}(\text{wage} > 250)$ as binary, logistic regression with a polynomial in age models

$$P(y_i > 250 \mid x_i) = \frac{\exp(\beta_0 + \beta_1 x_i + \cdots + \beta_d x_i^d)}{1 + \exp(\beta_0 + \beta_1 x_i + \cdots + \beta_d x_i^d)};$$

its confidence intervals (not drawn) are very wide because few people are high earners.

Step functions avoid imposing a global structure. Cut the range of X at $c_1 < \cdots < c_K$ into $K+1$ bins and use the indicators

$$C_0(X) = I(X < c_1),\ C_1(X) = I(c_1 \le X < c_2),\ \ldots,\ C_K(X) = I(c_K \le X),$$

which sum to 1 for every X. Fit $y_i = \beta_0 + \beta_1 C_1(x_i) + \cdots + \beta_K C_K(x_i) + \epsilon_i$. Then β_0 is the mean of Y for $X < c_1$, $\beta_0 + \beta_j$ the mean in $[c_j, c_{j+1})$, and β_j the average increase relative to the first bin.

```
coef(lm(wage ~ cut(age, 4), data = Wage))
```

```
##           (Intercept)   cut(age, 4)(33.5,49]   cut(age, 4)(49,64.5]
##             94.158392              24.053491              23.664559
## cut(age, 4)(64.5,80.1]
##              7.640592
```

Step functions miss trends within bins unless the cutpoints happen to be in the right places.

10.3 Regression splines

Motivation. Polynomials are smooth but global: a change in one region moves the whole curve. Step functions are local but jumpy. Splines combine the two virtues, fitting low-degree polynomials locally and joining them smoothly.

Basis functions. Polynomial and step regression are special cases of

$$y_i = \beta_0 + \beta_1 b_1(x_i) + \beta_2 b_2(x_i) + \cdots + \beta_K b_K(x_i) + \epsilon_i$$

with fixed, known basis functions chosen in advance: $b_j(x) = x^j$ for polynomials, $b_j(x) = I(c_j \le x < c_{j+1})$ for step functions. This is a standard linear model in the predictors $b_j(x_i)$; again we look at the fitted curve, not the coefficients.

Piecewise polynomials. Fit separate low-degree polynomials on different regions of X; the points where the coefficients change are the **knots**. A piecewise cubic with one knot at c:

$$y_i = \begin{cases} \beta_{01} + \beta_{11} x_i + \beta_{21} x_i^2 + \beta_{31} x_i^3 + \epsilon_i, & x_i < c, \\ \beta_{02} + \beta_{12} x_i + \beta_{22} x_i^2 + \beta_{32} x_i^3 + \epsilon_i, & x_i \ge c. \end{cases}$$

Unconstrained, the two pieces do not meet at c. Requiring continuity at c, and then continuity of the first and second derivatives, gives a curve that is continuous and smooth: a **cubic spline**. The left piece uses 4 degrees of freedom; the right piece 4 without constraints, 3 with continuity, 1 with the two derivative constraints as well. In general a cubic spline with K knots uses $K+4$ degrees of freedom.

Spline basis. A cubic spline with K knots can be written $y_i = \beta_0 + \beta_1 b_1(x_i) + \cdots + \beta_{K+3} b_{K+3}(x_i) + \epsilon_i$. The most direct basis starts from x, x^2, x^3 and adds one **truncated power function** per knot,

$$h(x, \xi) = (x - \xi)_+^3 = \begin{cases} (x - \xi)^3, & x > \xi, \\ 0, & \text{otherwise.} \end{cases}$$

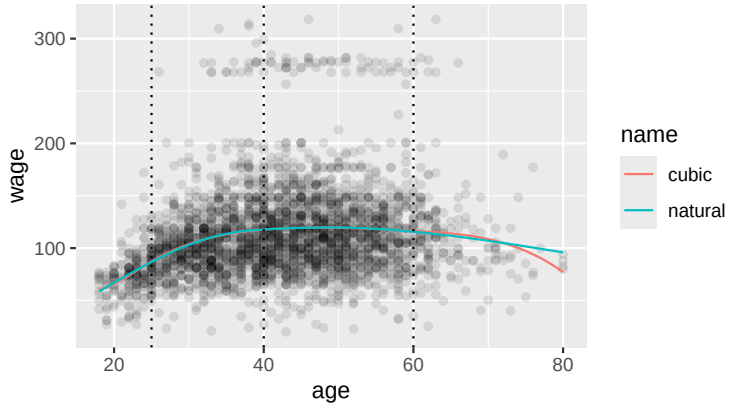
Adding $\beta h(x, \xi)$ to a cubic creates a discontinuity only in the third derivative at ξ, so the function and its first two derivatives stay continuous. With K knots: intercept, X, X^2, X^3 and $h(X, \xi_1), \ldots, h(X, \xi_K)$, i.e. $K+4$ coefficients.

Why the truncated power basis works [Supplement]

$h(x,\xi)$ and its first two derivatives, $3(x-\xi)_+^2$ and $6(x-\xi)_+$, vanish at $x=\xi$ from both sides, while the third derivative jumps from 0 to 6. Conversely, any cubic spline with knots $\xi_1 < \cdots < \xi_K$ differs on $[\xi_k, \xi_{k+1})$ from its expression on $[\xi_{k-1}, \xi_k)$ by a cubic that vanishes with two derivatives at ξ_k, which must be $c\,(x-\xi_k)^3$. So the $K+4$ functions span exactly the cubic splines.

In R, `bs()` gives a numerically better-behaved basis (B-splines) for the same space:

```
fit_bs <- lm(wage ~ bs(age, knots = c(25, 40, 60)), data = Wage)
fit_ns <- lm(wage ~ ns(age, df = 4), data = Wage)
grid <- tibble(age = 18:80) %>%
  mutate(cubic = predict(fit_bs, .), natural = predict(fit_ns, .))
ggplot(Wage, aes(age, wage)) + geom_point(alpha = 0.1) +
  geom_line(data = pivot_longer(grid, -age), aes(y = value, colour = name)) +
  geom_vline(xintercept = c(25, 40, 60), linetype = "dotted")
```



Natural splines. Splines have high variance near the ends of the range of X: the confidence bands there look wild. A **natural spline** adds boundary constraints, requiring the function to be linear beyond the boundary knots, which gives more stable estimates there (`ns()`).

Number and location of knots. Place more knots where the function might vary most rapidly; in practice knots are placed uniformly, e.g. at quantiles of X (`ns(age, df = 4)` does this). How many? Trying several and keeping the best-looking curve is subjective; cross-validation chooses the number that gives the smallest held-out RSS.

```
set.seed(1)
folds <- sample(rep(1:10, length.out = nrow(Wage)))
cv_ns <- function(df) mean(map_dbl(1:10, function(k) {
  fit <- lm(wage ~ ns(age, df = df), data = Wage[folds != k, ])
  mean((Wage$wage[folds == k] - predict(fit, Wage[folds == k, ]))^2)
}))
round(map_dbl(1:8, cv_ns), 1)
```

```
## [1] 1676.8 1602.4 1595.4 1595.2 1595.1 1594.0 1593.9 1593.6
```

CV error drops sharply up to about 3 degrees of freedom and is nearly flat after that. Compared with a polynomial of the same number of degrees of freedom, a natural spline is better behaved at the boundaries, because it uses knots rather than high powers to gain flexibility.

10.4 Smoothing splines

Motivation. Choosing knots is awkward. Smoothing splines avoid the choice by putting a knot at every data point and controlling wiggliness with a penalty instead, the same loss-plus-penalty idea as ridge regression.

We want a function g with small RSS $= \sum_i (y_i - g(x_i))^2$, but without constraints g can interpolate the data and make RSS zero (overfitting). We want g small in RSS *and* smooth. The **smoothing spline** minimises

$$\sum_{i=1}^{n} \left(y_i - g(x_i)\right)^2 + \lambda \int g''(t)^2 \, dt, \qquad \lambda \geq 0.$$

This is "loss + penalty": g'' measures roughness, since a smooth g has nearly constant slope and small $\int g''^2$, while a jumpy g has a wildly varying slope. With $\lambda = 0$ the penalty has no effect and g interpolates the training data; as $\lambda \to \infty$, g becomes a straight line. So λ controls the bias–variance trade-off. The minimiser is a natural cubic spline with knots at every unique x_i, shrunk by the penalty (the slides note the $\lambda = 0$ case).

Effective degrees of freedom. The fitted values are linear in y: $\hat{g}_\lambda = S_\lambda y$ for an $n \times n$ smoother matrix S_λ. The **effective degrees of freedom** are

$$\mathrm{df}_\lambda = \sum_{i=1}^{n} \{S_\lambda\}_{ii},$$

which decreases from n to 2 as λ goes from 0 to ∞ and measures flexibility. For least squares S is the hat matrix and its trace is the number of parameters, which motivates the definition [Supplement].

Choosing λ. No knots need to be chosen, but λ must be. LOOCV is cheap:

$$\mathrm{RSS}_{\mathrm{cv}}(\lambda) = \sum_{i=1}^{n} \left(y_i - \hat{g}_\lambda^{(-i)}(x_i)\right)^2 = \sum_{i=1}^{n} \left[\frac{y_i - \hat{g}_\lambda(x_i)}{1 - \{S_\lambda\}_{ii}}\right]^2,$$

where $\hat{g}_\lambda^{(-i)}$ is fitted without observation i; a single fit gives all n leave-one-out errors.

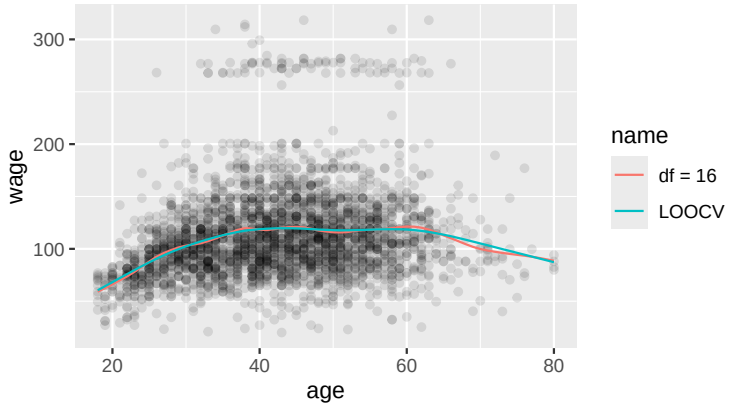
```
ss16 <- smooth.spline(Wage$age, Wage$wage, df = 16)
ss_cv <- smooth.spline(Wage$age, Wage$wage, cv = TRUE)
```

```
## Warning in smooth.spline(Wage$age, Wage$wage, cv = TRUE): cross-validation with non-unique 'x'
##     values seems doubtful
```

```
c(df_chosen = ss_cv$df, lambda = ss_cv$lambda)
```

```
##  df_chosen     lambda
## 6.79459570 0.02792303
```

```
tibble(age = 18:80, `df = 16` = predict(ss16, 18:80)$y,
       `LOOCV` = predict(ss_cv, 18:80)$y) %>%
  pivot_longer(-age) %>%
  ggplot(aes(age, value, colour = name)) +
  geom_point(data = Wage, aes(age, wage), inherit.aes = FALSE, alpha = 0.1) +
  geom_line() + labs(y = "wage")
```

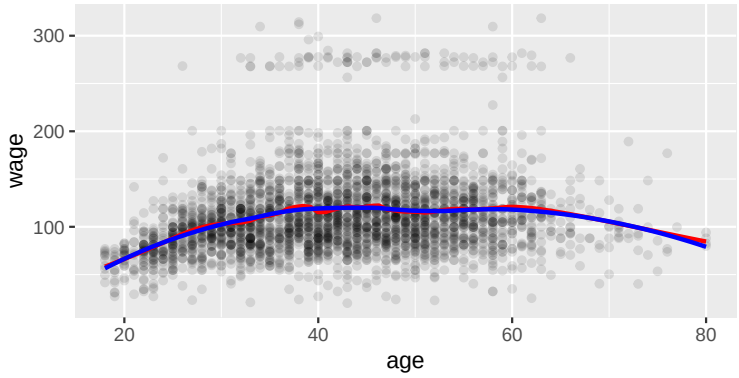


The LOOCV choice (about 7 effective degrees of freedom) is smoother than the 16-df fit and almost as good on the data.

10.5 Local regression

Local regression (LOESS, Lecture 2) fits at each target point x_0 a weighted least-squares line (or quadratic) using the fraction s (the **span**) of points nearest to x_0, with weights decreasing with distance. The span plays the role of λ: a smaller span gives a more wiggly fit. It is memory-based (all data are needed at prediction time) and works poorly in more than three or four dimensions.

```
ggplot(Wage, aes(age, wage)) + geom_point(alpha = 0.1) +
  geom_smooth(method = "loess", span = 0.2, se = FALSE, colour = "red", formula = y ~ x)
     +
  geom_smooth(method = "loess", span = 0.5, se = FALSE, colour = "blue", formula = y ~ x
    )
```



10.6 Generalized additive models

Motivation. All the smoothers so far handle one predictor. GAMs combine them: each predictor gets its own smooth function, and the functions are added, keeping the interpretability of an additive model.

A GAM replaces each linear term $\beta_j x_{ij}$ of the multiple regression model by a smooth function:

$$y_i = \beta_0 + \sum_{j=1}^{p} f_j(x_{ij}) + \epsilon_i = \beta_0 + f_1(x_{i1}) + f_2(x_{i2}) + \cdots + f_p(x_{ip}) + \epsilon_i.$$

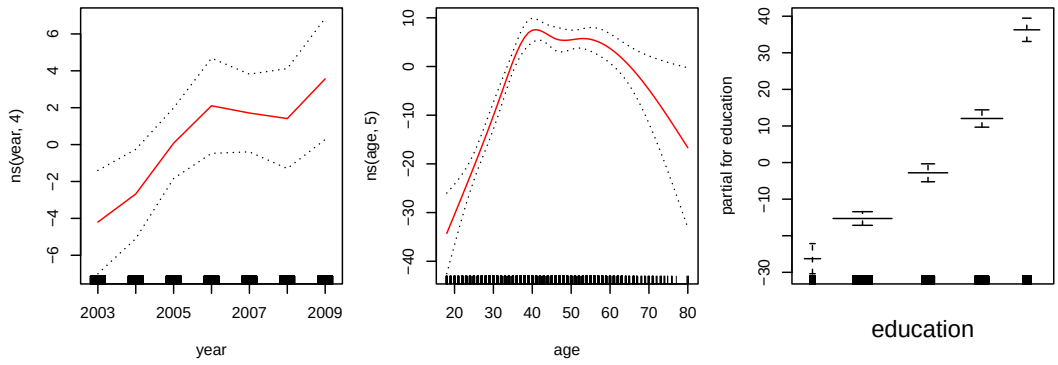
It is *additive* because a separate f_j is computed for each X_j and the contributions are added. The single-variable methods above are the building blocks, and GAMs work for quantitative and qualitative responses.

For the wage data,

$$\text{wage} = \beta_0 + f_1(\text{year}) + f_2(\text{age}) + f_3(\text{education}) + \epsilon,$$

with natural splines for year and age and a separate constant for each of the five education levels. Since natural splines are basis expansions, the whole model is one big regression on spline bases and dummy variables, fitted by `lm()`:

```
gam1 <- lm(wage ~ ns(year, 4) + ns(age, 5) + education, data = Wage)
par(mfrow = c(1, 3), mar = c(4, 4, 1, 1))
plot.Gam(gam1, se = TRUE, col = "red")
```



Each panel shows one f_j with the others held fixed: with age and education fixed, wage rises slightly with year; with education and year fixed, wage is highest in middle age; with age and year fixed, wage rises with education. Smoothing splines can be used instead (`gam::gam(wage ~ s(year, 4) + s(age, 5) + education)`, fitted by backfitting).

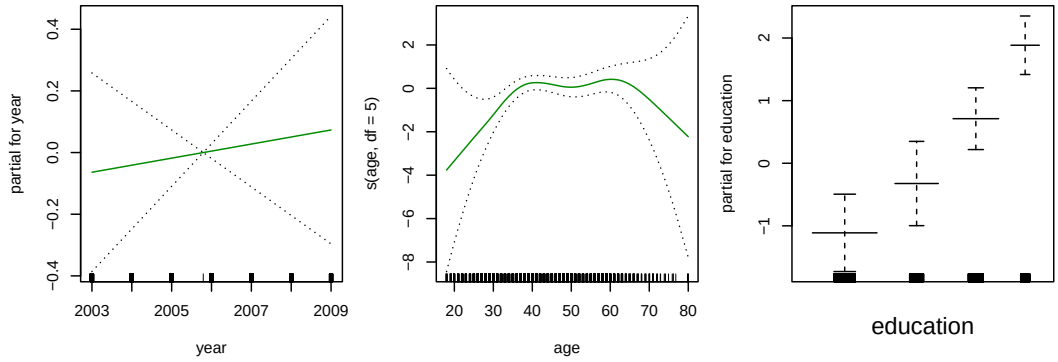
Pros. A nonlinear f_j is fitted to each X_j automatically, without trying transformations by hand; the fits can predict more accurately; additivity lets us examine each X_j's effect with the others fixed; the smoothness of each f_j is summarised by its degrees of freedom. **Cons.** The model is restricted to be additive, so important interactions can be missed. Interactions can be added by hand, as products $X_j \times X_k$ or as low-dimensional smooth functions $f_{jk}(X_j, X_k)$ (two-dimensional smoothers); fully general models need more flexible approaches such as random forests and boosting.

GAMs for classification. For binary Y with $p(X) = P(Y = 1 \mid X)$, the logistic GAM is

$$\log \frac{p(X)}{1 - p(X)} = \beta_0 + f_1(X_1) + f_2(X_2) + \cdots + f_p(X_p).$$

For the probability of earning more than \$250 000:

```
gam_lr <- gam(I(wage > 250) ~ year + s(age, df = 5) + education,
              family = binomial, data = Wage, subset = (education != "1. < HS Grad"))
par(mfrow = c(1, 3), mar = c(4, 4, 1, 1))
plot(gam_lr, se = TRUE, col = "green4")
```



The lowest education level is excluded because it contains no high earners, which would make its coefficient diverge (Lecture 9's separation problem) [Supplement].

Summary. Regression splines use basis functions; smoothing splines use loss plus penalty; GAMs extend linear models by replacing x_j with $f_j(x_j)$ while keeping additivity, and can use splines as building blocks.

Lecture 11. Causal Inference

11.1 Why causation

Motivation. Prediction asks what Y will be; many real questions ask what Y would be if we changed something. Answering that from data requires more than a good model: it requires assumptions about how treatment was assigned, and designs that make those assumptions believable.

Some questions are descriptive, but many, especially in medicine, are causal: Is smoking harmful? Would lower air pollution reduce deaths from respiratory disease? Does delivery at high-level neonatal intensive care units lower the risk of lung disease (BPD) as well as death? What proportion of fever cases is attributable to malaria?

A **confounder** X affects both the treatment Z and the outcome Y. Ice-cream sales and drownings rise together because hot weather drives both; often, however, we do not recognise all confounders.

Job training (LaLonde). A job-training programme in 1976: did it raise earnings in 1978? Treated and control people differ greatly before treatment:

```
lalonde %>%
  group_by(treat) %>%
  summarise(across(c(age, educ, married, re74, re75, re78), mean), black = mean(race ==
    "black"))
```

```
## # A tibble: 2 x 8
##   treat   age  educ married  re74  re75  re78 black
##   <int> <dbl> <dbl>   <dbl> <dbl> <dbl> <dbl> <dbl>
## 1     0  28.0  10.2   0.513 5619. 2466. 6984. 0.203
## 2     1  25.8  10.3   0.189 2096. 1532. 6349. 0.843
```

The treated are much more often Black, less often married, and earned far less in 1974–75. Is any difference in 1978 earnings due to the programme or to these factors?

Smoking and lung cancer. By the mid-1940s lung cancer had tripled in three decades, with candidate explanations including air pollution from cars and tar roads, an ageing population, radiography, and better diagnosis. In 1950 observational studies appeared (Wynder and Graham; Doll and Hill; Hammond). Hammond matched 36 975 heavy smokers to non-smokers on age, race, residence, occupational exposure, religion, education, marital status, alcohol, sleep, exercise and more. In 122 pairs exactly one person died of lung cancer: in 110 the smoker, in 12 the non-smoker. With no effect, each discordant pair is equally likely to go either way, so the number X of smoker deaths is $B(122, 1/2)$:

```
binom.test(110, 122, p = 0.5, alternative = "greater")$p.value
```

```
## [1] 2.733658e-21
```

The inference is simple; what could be wrong is an *unmeasured* confounder. R. A. Fisher, a heavy smoker, argued that a gene might cause both smoking and cancer. Association is not causation, but causal reasoning is hard.

How people reason about causes. Zeus receives a heart transplant and dies five days later. If we could know that without the transplant he would have been alive after five days, we would say the transplant caused his death. Causal effects compare what happened with what *would have happened*.

11.2 Potential outcomes

Motivation. The comparison "with versus without treatment" needs notation in which both possibilities exist. Potential outcomes provide it, and they make precise both what we want to estimate and why it cannot be observed directly.

For units $i = 1, \ldots, N$: covariates X_i; treatment $Z_i \in \{0, 1\}$; **potential outcomes** $Y_i(1)$ (if treated) and $Y_i(0)$ (if not); and the observed outcome

$$Y_i = Y_i(Z_i) = Z_i Y_i(1) + (1 - Z_i) Y_i(0).$$

For Zeus, $Z_i = 1$ and $Y_i = 1$ (death), with $Y_i(1) = 1$, $Y_i(0) = 0$: $Y_i(1)$ is the *factual* outcome and $Y_i(0)$ the *counterfactual* one, which can only be imagined. (Other texts write treatment A and potential outcomes $Y^{a=1}$, $Y^{a=0}$.)

The notation hides assumptions:

- **no interference**: Y_i depends only on unit i's own treatment, not on Z_j or Y_j for $j \neq i$;
- **consistency**: $Y_i = Y_i(1)$ if $Z_i = 1$, which requires **no multiple versions of treatment**: all treated units receive the same treatment (e.g. placebo patients must receive nothing else).

Together these are the **stable unit treatment value assumption** (SUTVA).

Causal effects. The individual effect is $\tau_i = Y_i(1) - Y_i(0)$; the **average treatment effect** is

$$\tau = \mathbb{E}[Y_i(1) - Y_i(0)] = \mathbb{E}[Y_i(1)] - \mathbb{E}[Y_i(0)].$$

Other measures: the risk ratio $\mathbb{E}[Y_i(1)]/\mathbb{E}[Y_i(0)]$ and, for binary outcomes, the odds ratio $\dfrac{P(Y_i(1) = 1)\, P(Y_i(0) = 0}{P(Y_i(1) = 0)\, P(Y_i(0) = 1}$

The **sample** ATE is $\frac{1}{N} \sum_i \{Y_i(1) - Y_i(0)\}$, the **population** ATE is $\mathbb{E}[Y_i(1) - Y_i(0)]$, and SATE $\to$ PATE as $N \to \infty$.

Science table versus statistician's table. The "science table" lists both potential outcomes and the covariates for every unit; the data reveal only $Y_i(1)$ for treated units and $Y_i(0)$ for controls, the other entry being "?" (covariates are fully observed unless missing).

The fundamental problem of causal inference (Holland, 1986): τ_i is never observed, since only one potential outcome is. Individual effects cannot be estimated, but under assumptions on the treatment assignment the ATE $\mathbb{E}[\tau_i] = \mathbb{E}[Y_i(1)] - \mathbb{E}[Y_i(0)]$ can: random assignment (randomised experiments) or unconfoundedness (observational studies), together with **overlap**, $0 < P(Z = 1 \mid X) < 1$: every unit has a chance of receiving and of not receiving treatment.

11.3 Randomised experiments and observational studies

Motivation. Randomisation makes the treated and control groups alike in every respect, measured or not, so their difference in means is the causal effect. Without randomisation, we must create comparable groups ourselves, using the covariates.

Randomised experiments (clinical trials, A/B tests) are the gold standard but often infeasible; observational studies (census, shopping histories, streaming data) are easy to obtain but hard to analyse.

Under random assignment, treated and control groups tend to be similar in both observed and unobserved covariates; randomisation gives unbiased estimators and is the "reasoned basis for inference" (Fisher, 1935), since the physical act of randomising induces the distribution used for exact tests. Formally, random assignment gives

$$(Y(1), Y(0)) \perp\!\!\!\perp Z.$$

In a cholesterol-drug trial with assignment by a fair coin, people's potential cholesterol levels have nothing to do with the coin, so $\mathbb{E}[Y(1)] = \mathbb{E}[Y(1) \mid Z = 1] = \mathbb{E}[Y(1) \mid Z = 0]$, and $Y(1)$ is observed when $Z = 1$. Hence

$$\mathbb{E}[Y(1) - Y(0)] = \mathbb{E}[Y(1) \mid Z = 1] - \mathbb{E}[Y(0) \mid Z = 0] = \mathbb{E}[Y \mid Z = 1] - \mathbb{E}[Y \mid Z = 0] = \bar{Y}_t - \bar{Y}_c,$$

the difference in group means, with no assumption on the distributions of $Y(1), Y(0)$. Independence of potential outcomes and treatment is the **identification assumption**.

When it fails. If a doctor gives the drug to patients expected to have high cholesterol without it, higher $Y_i(0)$ means a higher chance of $Z_i = 1$, so $\mathbb{E}[Y(0) \mid Z = 1] > \mathbb{E}[Y(0) \mid Z = 0]$ and the naive comparison is biased. The doctor's decision is based on medical history X, a confounder.

In an observational study treated and control groups differ systematically. **Overt bias** from observed covariates is removed by adjustment (regression, matching, weighting); **hidden bias** from unobserved covariates needs special designs (instrumental variables, discontinuities, difference-in-differences) and sensitivity analysis.

11.3.1 Unconfoundedness

The key assumption is

$$(Y(1), Y(0)) \perp\!\!\!\perp Z \mid X \qquad \text{(unconfoundedness, ignorability)} :$$

given enough information X, treatment looks random among units with the same X. Then

$$\mathbb{E}[Y(1)] = \mathbb{E}\big[\mathbb{E}[Y(1) \mid X]\big] = \mathbb{E}\big[\mathbb{E}[Y(1) \mid X, Z = 1]\big] = \mathbb{E}\big[\mathbb{E}[Y \mid X, Z = 1]\big],$$

and likewise for $Y(0)$ with $Z = 0$. So fit an outcome model on the treated, $\hat{\mu}_1(X)$, another on the controls, $\hat{\mu}_0(X)$, and estimate

$$\hat{\tau}^{\text{ATE}} = \frac{1}{n} \sum_{i=1}^{n} \{\hat{\mu}_1(X_i) - \hat{\mu}_0(X_i)\}.$$

In general $\mathbb{E}[Y(1)] \neq \mathbb{E}[Y \mid Z = 1]$: causation and association differ.

Connection to regression. The classical adjustment $\mathbb{E}(Y \mid X) = \beta_0 + \beta_1 Z + \beta_2 X$ corresponds to $\mathbb{E}[Y(1) \mid X] = \beta_0 + \beta_1 + \beta_2 X$ and $\mathbb{E}[Y(0) \mid X] = \beta_0 + \beta_2 X$, so the ATE is β_1. But the two conditional means may need very different specifications, and $\mathbb{E}[Y(1) \mid X] - \mathbb{E}[Y(0) \mid X] = \tau(X)$ is in general a function of X. Concerns: lack of overlap leads to *extrapolation* (if no treated unit has a given X, what does the comparison mean?), and covariate adjustment is usually done while looking at outcomes, which invites cherry-picking.

11.4 Propensity score methods

Motivation. Matching people on many covariates at once is hopeless, since nobody shares all values. The propensity score compresses the covariates into a single number, the probability of treatment, and comparing people with the same score is enough.

The **propensity score** is $e(X_i) = P(Z_i = 1 \mid X_i)$, a one-dimensional summary of possibly many covariates (Rosenbaum and Rubin, 1983). Key result:

Theorem 11.1 (balancing property)

If $(Y(1), Y(0)) \perp\!\!\!\perp Z \mid X$, then $(Y(1), Y(0)) \perp\!\!\!\perp Z \mid e(X)$.

Proof [Supplement]. It suffices to show $P(Z = 1 \mid Y(1), Y(0), e(X)) = e(X)$, which does not depend on the potential outcomes. By the tower property, $P(Z = 1 \mid Y(1), Y(0), e(X)) = \mathbb{E}\big[P(Z = 1 \mid Y(1), Y(0), X) \mid Y(1), Y(0), e(X)\big] = \mathbb{E}\big[e(X) \mid Y(1), Y(0), e(X)\big] = e(X)$, using unconfoundedness in the middle step and that $e(X)$ is a function of X. ■

So we need not find people sharing all covariate values; sharing $e(X)$ is enough. The propensity score is known in a randomised experiment but must be estimated in an observational study, usually by logistic regression, $\operatorname{logit} P(Z_i = 1 \mid X_i) = \beta_X^\mathsf{T} X_i$ and $\hat{e}(X_i) = \operatorname{expit}(\hat{\beta}_X^\mathsf{T} X_i)$. It is then used for stratification, matching or weighting.

11.4.1 Stratification

Within strata of units with the same $e(X)$, unconfoundedness makes treatment random. Since $\hat{e}$ is continuous, make narrow strata: (1) estimate $\hat{e}(X_i)$; (2) cut it into K strata, e.g. at quantiles or at $(0, \frac{1}{K}], \ldots, (\frac{K-1}{K}, 1)$; (3) treat each stratum as a randomised experiment:

$$\hat{\tau} = \sum_{k=1}^{K} (\bar{Y}_{k1} - \bar{Y}_{k0}) \, \frac{n_k}{n},$$

with $\bar{Y}_{k1}$, $\bar{Y}_{k0}$ the treated and control means in stratum k of size n_k. Equivalently, a treated unit in stratum k has weight $\frac{1}{n_{k1}} \frac{n_k}{n}$ and a control $\frac{1}{n_{k0}} \frac{n_k}{n}$, and a weighted regression $Y = \alpha + \beta Z + \epsilon$ gives $\hat{\tau} = \hat{\beta}$.

11.4.2 Matching

1:1 matching, when there are fewer treated than controls: (1) for each treated unit select a control with the same (or similar) X or $e(X)$, with or without replacement (a bias–variance trade-off); (2) check covariate balance between the treated and matched controls; (3) discard unmatched controls, which often reduces variance; (4) analyse the pairs as a paired randomised experiment (like twins): the mean of treated-minus-control differences estimates the effect (or regress Y on Z in the matched data; adding X is possible but not recommended), and randomisation inference tests the null hypothesis of no effect.

11.4.3 Inverse probability weighting

In a randomised trial the groups are balanced; in an observational study they are not. **IPW** weights treated units by $1/e(X)$ and controls by $1/(1 - e(X))$. Intuition: a treated person with $e(X) = 1/5$ stands for five "clones", one treated and four controls on average, so he represents all five. The weights create a **pseudo-population** in which treatment no longer depends on X: unconfoundedness given X in the original population becomes $(Y(1), Y(0)) \perp\!\!\!\perp Z$ in the pseudo-population.

Proposition 11.2 (IPW identification)

Under unconfoundedness and overlap,

$$\tau = \mathbb{E}\Big[\frac{Z_i Y_i}{e(X_i)}\Big] - \mathbb{E}\Big[\frac{(1 - Z_i) Y_i}{1 - e(X_i)}\Big].$$

Proof. First term: $\mathbb{E}\big[\frac{Z_i Y_i}{e(X_i)}\big] = \mathbb{E}\big[\frac{\mathbb{E}[Z_i Y_i \mid X_i]}{e(X_i)}\big] = \mathbb{E}\big[\frac{e(X_i)\,\mathbb{E}[Y_i \mid Z_i = 1, X_i]}{e(X_i)}\big] = \mathbb{E}[\mathbb{E}[Y_i(1) \mid Z_i = 1, X_i]] = \mathbb{E}[\mathbb{E}[Y_i(1) \mid X_i]] = \mathbb{E}[Y_i(1)]$, using $\mathbb{E}[Z_i Y_i \mid X_i] = P(Z_i = 1 \mid X_i)\mathbb{E}[Y_i \mid Z_i = 1, X_i]$, consistency and unconfoundedness. The second term equals $\mathbb{E}[Y_i(0)]$ in the same way [Supplement]. ■

With estimated scores $\hat{e}_i$,

$$\hat{\tau}^{\text{ATE}} = \frac{1}{n}\sum_i \frac{Z_i Y_i}{\hat{e}_i} - \frac{1}{n}\sum_i \frac{(1-Z_i)Y_i}{1-\hat{e}_i},$$

or a regression of Y on Z with weights $w_i = Z_i/\hat{e}_i + (1-Z_i)/(1-\hat{e}_i)$. When $\hat{e}_i$ is near 0 or 1 the weights are huge and the estimate unstable. For the effect on the treated (ATT) the weights are 1 for treated units and $\hat{e}_i/(1-\hat{e}_i)$ for controls.

11.5 Example: job training and earnings

`MatchIt` (matching) and `WeightIt` (weighting) implement these methods; `cobalt::bal.tab()` checks balance and `marginaleffects::avg_comparisons()` estimates effects with appropriate standard errors. The data are `cobalt::lalonde` (185 treated, 429 controls).

```
no.adj <- matchit(treat ~ age + educ + race + married + nodegree + re74 + re75,
                  data = lalonde, method = NULL, distance = "glm")
summary(no.adj)$sum.all[, 1:3]
```

```
##            Means Treated Means Control Std. Mean Diff.
## distance    5.774355e-01     0.1822248      1.79408605
## age         2.581622e+01    28.0303030     -0.30944526
## educ        1.034595e+01    10.2354312      0.05496466
## raceblack   8.432432e-01     0.2027972      1.76154189
## racehispan  5.945946e-02     0.1421911     -0.34984254
## racewhite   9.729730e-02     0.6550117     -1.88186760
## married     1.891892e-01     0.5128205     -0.82630927
## nodegree    7.081081e-01     0.5967366      0.24497023
## re74        2.095574e+03  5619.2365064     -0.72108381
## re75        1.532055e+03  2466.4844431     -0.29026291
```

Standardised mean differences before adjustment are huge (1.76 for Black, -1.88 for White, -0.72 for 1974 earnings); a common rule of thumb asks for $|\text{SMD}| < 0.1$ [Supplement]. Three adjustments:

```
ps.str <- matchit(treat ~ age + educ + race + married + nodegree + re74 + re75,
                  data = lalonde, method = "subclass", distance = "glm", link = "probit"
    , subclass = 7)
ps.mat <- matchit(treat ~ age + educ + race + married + nodegree + re74 + re75,
                  data = lalonde, method = "nearest", distance = "glm", link = "probit",
     replace = TRUE)
W <- weightit(treat ~ age + educ + race + married + nodegree + re74 + re75,
              data = lalonde, method = "glm", estimand = "ATT")
round(cbind(stratified = summary(ps.str, un = FALSE)$sum.across[, 3],
            matched = summary(ps.mat, un = FALSE)$sum.matched[, 3]), 3)
```

```
##            stratified matched
## distance        0.063   0.003
## age            -0.036  -0.085
## educ            0.027  -0.083
## raceblack       0.084   0.015
## racehispan     -0.018   0.046
## racewhite      -0.089  -0.055
## married        -0.030   0.097
## nodegree        0.027   0.143
## re74           -0.089  -0.013
## re75           -0.074  -0.017
```

```
bal.tab(W, un = TRUE)
```

```
## Balance Measures
##                 Type Diff.Un Diff.Adj
## prop.score  Distance  1.7941  -0.0205
## age          Contin. -0.3094   0.1188
## educ         Contin.  0.0550  -0.0284
```

```
## race_black    Binary  0.6404  -0.0022
## race_hispan   Binary -0.0827   0.0002
## race_white    Binary -0.5577   0.0021
## married       Binary -0.3236   0.0186
## nodegree      Binary  0.1114   0.0184
## re74         Contin. -0.7211  -0.0021
## re75         Contin. -0.2903   0.0110
##
## Effective sample sizes
##            Control Treated
## Unadjusted  429.       185
## Adjusted     99.82     185
```

All three bring the covariates close to balance. Stratification uses 7 strata; 1:1 nearest-neighbour matching with replacement uses only 84 distinct controls (the others are discarded); IPW keeps everyone with weights. Effects on the treated, with a weighted regression of 1978 earnings on treatment:

```
str.data <- match.data(ps.str); mat.data <- match.data(ps.mat)
est <- function(fit, ...) {
  a <- avg_comparisons(fit, variables = "treat", ...)
  round(c(estimate = a$estimate, low = a$conf.low, high = a$conf.high))
}
fit.str <- lm(re78 ~ treat, data = str.data, weights = weights)
fit.mat <- lm(re78 ~ treat, data = mat.data, weights = weights)
lalonde$ipw.weight <- W$weights
fit.ipw <- lm(re78 ~ treat, data = lalonde, weights = ipw.weight)
rbind(
  stratification = est(fit.str, vcov = ~subclass, newdata = subset(str.data, treat == 1)
    , wts = "weights"),
  matching       = est(fit.mat, vcov = "HC3", newdata = subset(mat.data, treat == 1),
    wts = "weights"),
  ipw            = est(fit.ipw, vcov = "HC3", wts = "ipw.weight"),
  naive          = round(c(coef(lm(re78 ~ treat, data = lalonde))[2], NA, NA)))
```

```
##               estimate  low high
## stratification     1301 -423 3025
## matching           1290 -603 3184
## ipw                1214 -416 2844
## naive              -635   NA   NA
```

Full matching (each unit in a matched set, no units discarded) gives a larger estimate:

```
ps.full <- matchit(treat ~ age + educ + race + married + nodegree + re74 + re75,
                   data = lalonde, method = "full", distance = "glm", link = "probit")
full.data <- match.data(ps.full)
fit.full <- lm(re78 ~ treat, data = full.data, weights = weights)
est(fit.full, vcov = ~subclass, newdata = subset(full.data, treat == 1), wts = "weights"
    )
```

```
## estimate      low     high
##     1855      364     3346
```

The naive comparison suggests the programme *lowered* earnings; after adjustment the estimates are positive, around \$1 200–1 900. The original randomised experiment found \$2 114 with 95% CI (458, 3955), closest to full matching.

Take-home messages. Matching and weighting separate *design* from *analysis*: neither uses the outcome at the design stage. Matching finds similar subjects with respect to X or $e(X)$; weighting uses inverse probabilities of the treatment received. The analysis is then simple: compare means, or fit a simple regression. Try several methods and check whether they agree.